

LABORATORIO
03



FACULTAD DE INGENIERÍA DE MINAS, GEOLOGÍA Y CIVIL
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE MINAS Y CIVIL

MÉTODOS NUMÉRICOS

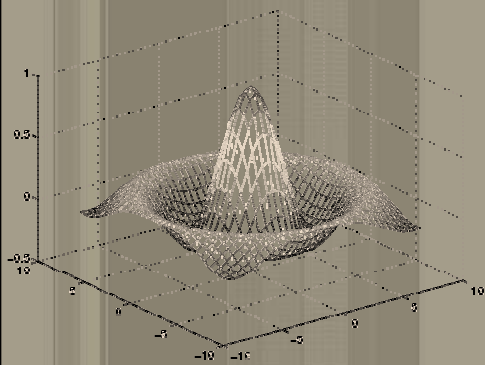
Ingeniería Civil



ING. CRISTIAN CASTRO P.

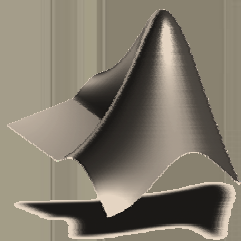
Laboratorio de Computación Numérica

ING. CRISTIAN CASTRO P.

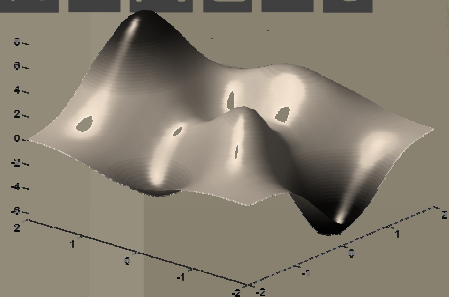


Sesión 03

SISTEMAS DE ECUACIONES ALGEBRAICAS LINEALES



- Métodos Numéricos
- Ingeniería Civil



Sistema de Ecuaciones Algebraicas Lineales

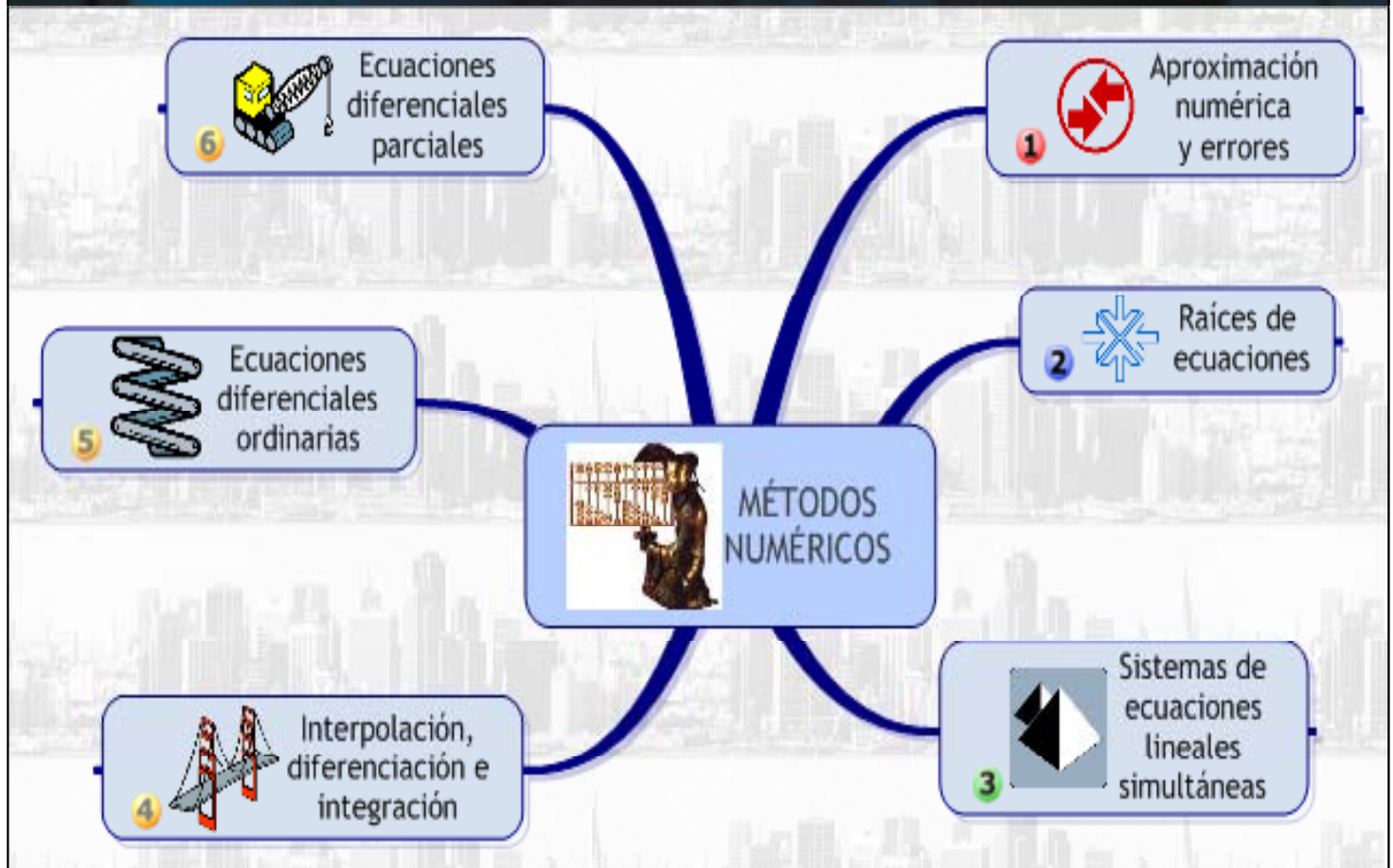
Ingeniería Civil – FIMGC – UNSCH

cristiancastro cristiancastro cristiancastro cristiancastro cristiancastro cristiancastro cristiancastro cristiancastro cristiancastro cristiancastro



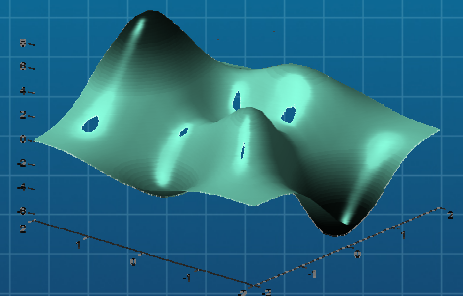
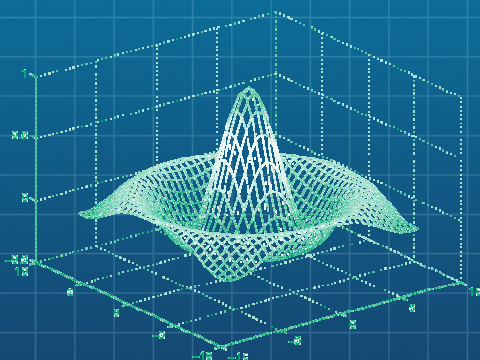
ING. CRISTIAN CASTRO P.

MÉTODOS NUMÉRICOS



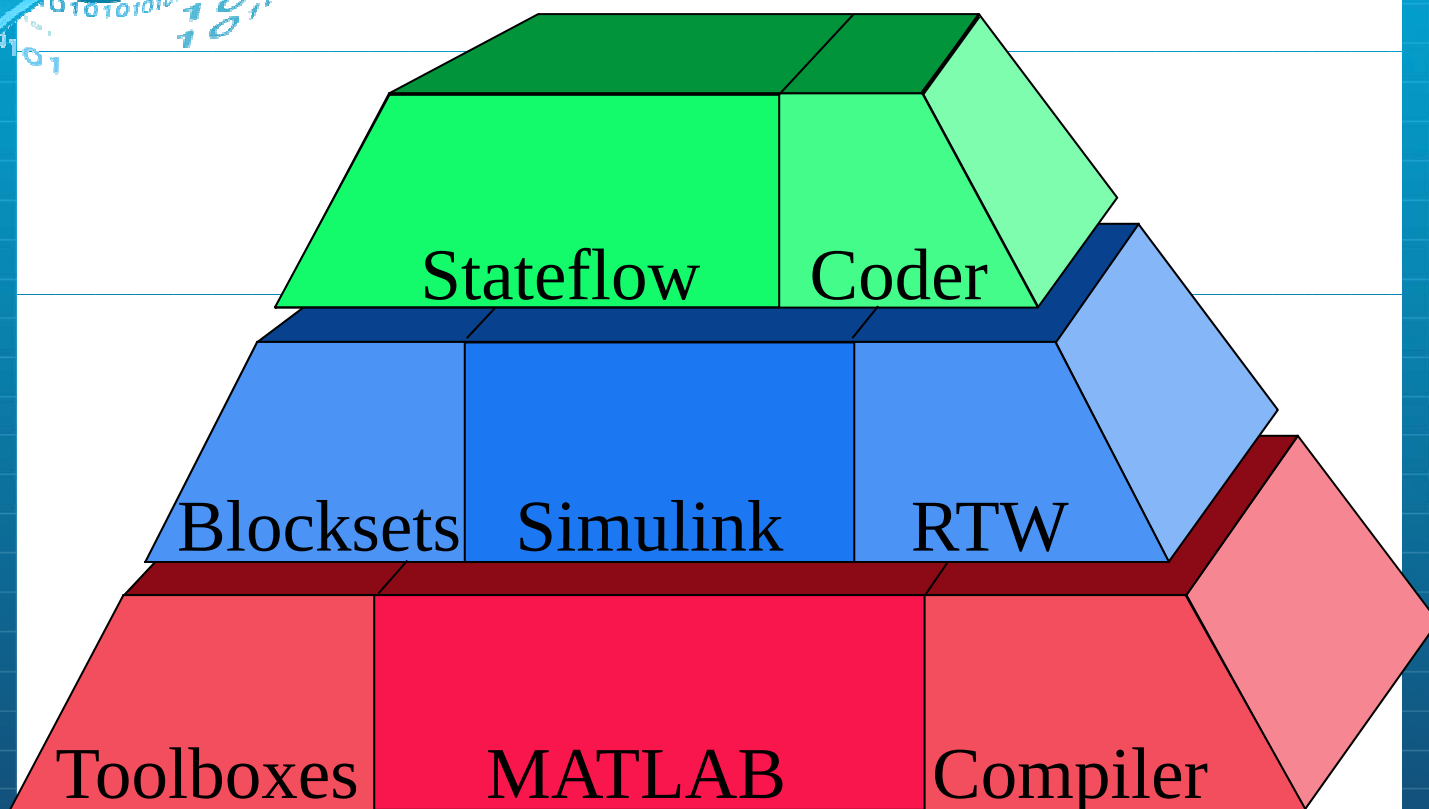
Consideraciones de Matrices en MATLAB

Métodos Numéricos
Aplicados a la Ingeniería



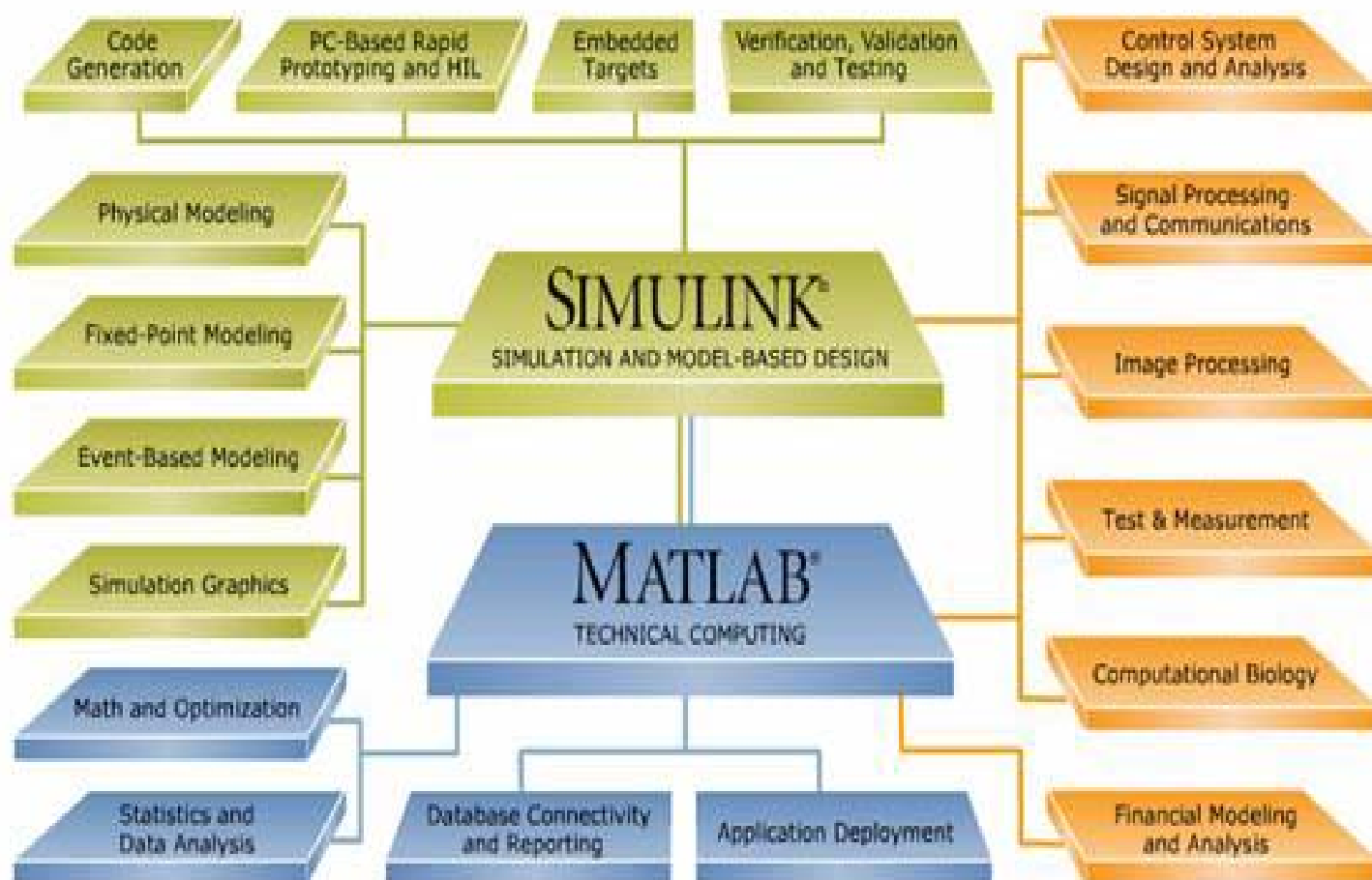


The MathWorks Product Suite



Simulink Product Family

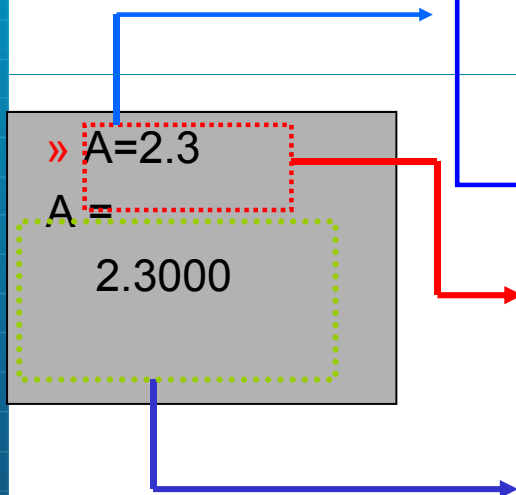
Application-Specific Products





MATLAB: Asignación

- Variable names:
 - Starts with a letter
 - Up to 31 characters (some use 19 or 21)
 - May contain letters, digits and underscore_
 - Case sensitive ("A" is not the same as "a")



The MATLAB command

This is the result of the MATLAB statement



MATLAB: Asignación Escalar

```
» A=2.3
A =
    2.3000
```

this creates a variable "A"
and set its value to 2.3

```
» A=[2.3]
A =
    2.3000
```

The square braces **[]** are
used to define matrices.
We can use them for
scalars too.



MATLAB: Vector Fila

The square braces are used to define a matrix

```
» X=[2,3 7]
```

X =

2 3 7

Space or comma are used to separate elements in the same row



MATLAB: Vector Columna

The square braces are used to define a matrix

```
» X=[2;3 ; 7]
```

X =

2

3

7

semicolon are used to end a row.

You can also use ENTER to end a row



MATLAB: Declaraciones

MATLAB Statement	Remarks
<code>C=5.66</code>	C is a scalar
<code>C=[5.66]</code>	An alternative way
<code>X=[3.5 6.3, 33]</code>	X is a 1X3 matrix with elements 3.5 , 6.3 and 33. Commas or space are used to separate the elements in a row
<code>Y=[1 4]</code>	Y is a 2X1 matrix whose elements are 1 and 4.
<code>Y = [1 ; 4]</code>	Semicolon are used to indicate the end of the row.
<code>A=1:5</code>	Equivalent to <code>A=[1 2 3 4 5]</code>



MATLAB: Declaraciones

MATLAB Statement	Remarks
<code>V=[2 3 5 3 3 8]</code>	$V = \begin{bmatrix} 2 & 3 & 5 \\ 3 & 3 & 8 \end{bmatrix}$
<code>C=[1:3:11]</code>	<code>C=[1 4 7 10]</code>
<code>Z=4\8</code>	<code>Z=2</code>
<code>Y=eye(2)</code>	$V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$
<code>W = zeros(2,3)</code>	$V = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$



Ejemplo: Matriz Mágica

A =

16	3	2	13
5	10	11	8
9	6	7	12
4	15	14	1

A=magic(4)

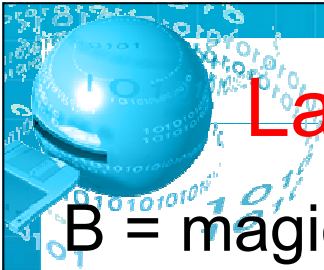
sum(A) se obtienen las sumas de las columnas

sum(A') se obtienen las sumas de las filas

sum(diag(A)) se obtiene la suma de la diagonal principal

sum(diag(rot90(A))) se obtiene la suma de la otra diagonal

fliplr(A) ==> sum(diag(fliplr(A)))



La Función “magic”

B = magic(4)

B =

16	2	3	13
5	11	10	8
9	7	6	12
4	14	15	1

The Colon (:) Operator

1:10 ==> [1 2 3 4 5 6 7 8 9 10]

0:10:50 ==> [0 10 20 30 40 50]

0:pi/4:pi ==> [0 0.7854 1.5708 2.3562 3.1416]



VECTORES Y MATRICES

- Las matrices son el tipo fundamental de dato en Matlab.

```
» A=[1 3 5; 6 9 2; 4 8 7]
```

A =

1	3	5
6	9	2
4	8	7

```
» det(A)
```

ans =

5

```
» A^2+3*A
```

ans =

42	79	61
86	142	68
92	164	106

MATrix LABoratory

-- datos son matrices

-- reglas del álgebra lineal



La matriz en MatLab

		Columnas (n)				
		1	2	3	4	5
Filas (m)	1	4 ¹	10 ⁶	1 ¹¹	6 ¹⁶	2 ²¹
	2	8 ²	1.2 ⁷	9 ¹²	4 ¹⁷	25 ²²
	3	7.2 ³	5 ⁸	7 ¹³	1 ¹⁸	11 ²³
	4	0 ⁴	0.5 ⁹	4 ¹⁴	5 ¹⁹	56 ²⁴
	5	23 ⁵	83 ¹⁰	13 ¹⁵	0 ²⁰	10 ²⁵

A (2,4)

A (17)

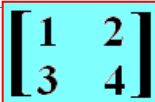
Matriz rectangular:

Escalar: matriz de 1X1

Vector: matriz de mX1
matriz de 1Xn

Matriz: matriz de mXn

Creación de matrices numéricas


$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

```
» a=[1 2;3 4]
```

```
a =
```

```
1    2
3    4
```

corchetes rectangulares

```
» b=[-2.8, sqrt(-7), (3+5+6)*3/4]
```

```
b =
```

```
-2.8000    0 + 2.6458i    10.5000
```

```
» b(2,5) = 23
```

```
b =
```

```
-2.8000    0 + 2.6458i    10.5000    0    0
0    0    0    0    23.0000
```

NOTA:

- 1) Separador de fila
punto y coma (;)
- 2) Separador de columna
espacio o coma (,)

1. Cada expresión de MatLab puede ser ingresada como un elemento de una matriz (internamente es otra matriz)
2. En MatLab, los matrices siempre son rectangulares

Expansión Escalar

Expansión escalar:

Suma de matriz + escalar

```
» w=[1 2;3 4] + 5
```

```
w =
```

```
6    7
8    9
```

Creación de secuencias:

operador punto y coma (:)

```
» x = 1:5
```

```
x =
```

```
1    2    3    4    5
```

```
» y = 2:-0.5:0
```

```
y =
```

```
2.0000    1.5000    1.0000    0.5000    0
```

Función de uso práctico para crear matrices.

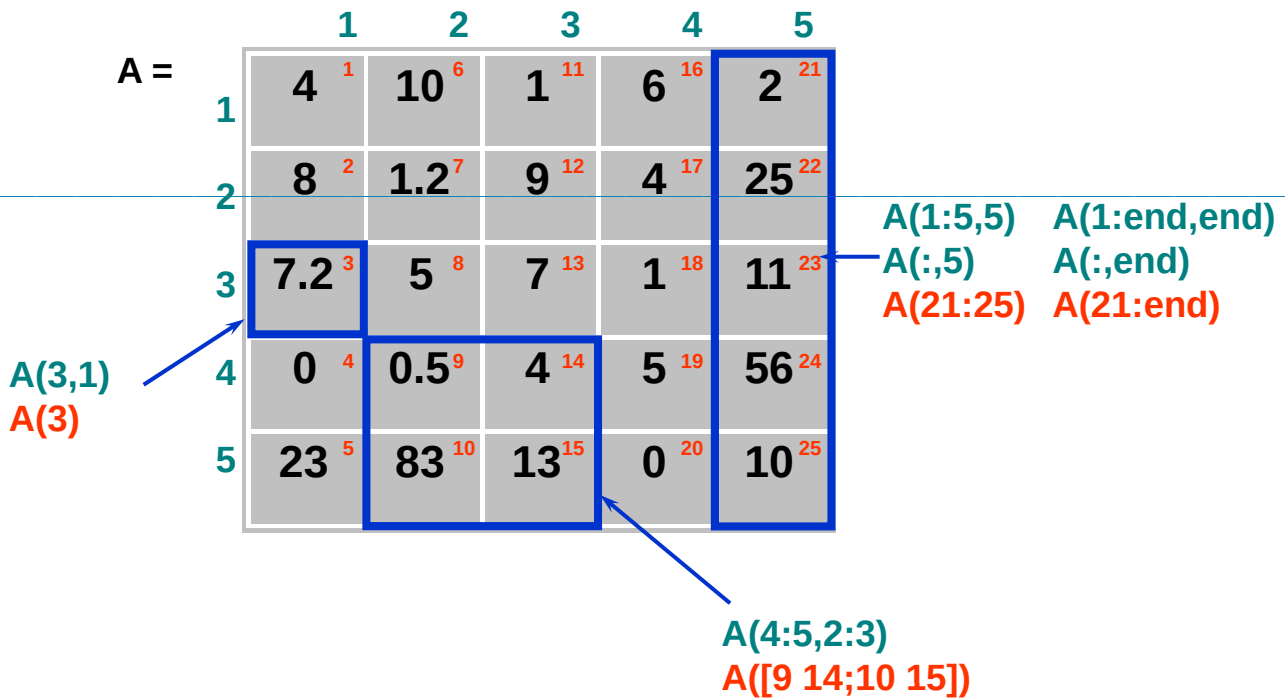
```
» z = rand(2,4)
```

```
z =
```

```
0.9501    0.6068    0.8913    0.4565
0.2311    0.4860    0.7621    0.0185
```



Extracción desde un matriz



Concatenación de matrices

Use [] para combinar los matrices como "elementos" de una matriz

Separador de fila
punto y coma (;)

Separador de columna
espacio o coma (,)

```

>> a=[1 2;3 4]
a =
     1     2
     3     4
>> cat_a=[a, 2*a; 3*a, 4*a; 5*a, 6*a]
cat_a =
     1     2     2     4
     3     4     6     8
     3     6     4     8
     9    12    12    16
     5    10     6    12
    15    20    18    24
    
```

← corchetes rectangulares []

← 4*a

La matriz resultante siempre es rectangular

Borrar filas o columnas

```
» A=[1 5 9;4 3 2.5; 0.1 10 3i+1]
```

```
A =
```

1.0000	5.0000	9.0000
4.0000	3.0000	2.5000
0.1000	10.0000	1.0000+3.0000i

```
» A(:,2)=[]
```

```
A =
```

1.0000	9.0000
4.0000	2.5000
0.1000	1.0000 + 3.0000i

```
» A(2,2)=[]
```

??? Indexed empty matrix assignment is not allowed.

Multiplicación de matrices

```
» a = [1 2 3 4; 5 6 7 8];
```

```
» b = ones(4,3);
```

```
» c = a*b
```

```
c =
```

10	10	10
26	26	26

[2x4] [4x3] → [2x3]

← a(2da fila).b(3ra columna)

Multiplicación punto

```
» a = [1 2 3 4; 5 6 7 8];
```

```
» b = [1:4; 1:4];
```

```
» c = a.*b
```

```
c =
```

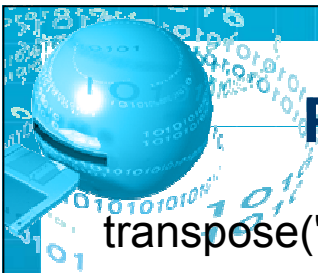
1	4	9	16
5	12	21	32

← c(2,4) = a(2,4)*b(2,4)



Funciones para manipular matrices

zeros:	Crea un matriz de ceros
ones:	Crea un matriz de unos
eye:	Matriz identidad
rand:	Números aleatorios uniformemente distribuidos
diag:	Matriz diagonal y diagonal de una matriz
size:	Dimensiones de la matriz
fliplr:	Invierte la matriz de derecha a izquierda
flipud:	Invierte la matriz de arriba hacia abajo
repmat:	Replica la matriz



Funciones para manipular matrices

transpose('):	Transpuesta de la matriz
rot90:	Girar la matriz 90°
tril:	La parte triangular baja de una matriz
triu:	La parte triangular superior de una matriz
cross:	Producto cruz de vectores
dot:	Producto punto de vectores
det:	Determinante de la matriz
inv:	Inversa de la matriz
eig:	Calcula los eigenvalues y eigenvectors.
rank:	Rango de la matriz

Si $\mathbf{a} = (1,2,3)$ y $\mathbf{b} = (4,5,6)$, el producto cruz $\mathbf{a} \times \mathbf{b}$ es:

$$\mathbf{a} \times \mathbf{b} = (1,2,3) \times (4,5,6) = ((2 \times 6 - 3 \times 5), -(1 \times 6 - 3 \times 4), +(1 \times 5 - 2 \times 4)) = (-3,6,-3).$$



Funciones de Matlab para manejo de matrices

- **size** devuelve el número de filas y de columnas de una matriz
- **length** devuelve la longitud de un vector o la máxima dimensión de una matriz
- **inv(A)** calcula la inversa de la matriz A
- **A'** es la transpuesta de la matriz A
- **d=eig(A)** devuelve los valores propios asociados con la matriz A cuadrada como un vector columna
- **[V,D]=eig(A)** devuelve vectores propios en la matriz V y los valores propios como elementos diagonales en la matriz D
- **rank(A)** devuelve el rango de la matriz A
- **norm(A)** Calcula la norma de la matriz A. admite el cálculo de norma -1, norma-2, norma-∞
- **poly(A)** encuentra el polinomio característico asociado con la matriz cuadrada A
- **flipud(A)** Intercambia una matriz de arriba a bajo
- **fliplr(A)** Intercambia una matriz de izquierda a derecha
- **rot90(A)** gira una matriz en dirección contraria a las manecillas del reloj
- **diag(v)** crea una matriz diagonal, con el vector **v** sobre la diagonal
- **diag(A)** extrae la diagonal de la matriz **A** como un vector columna



Matrices especiales

Podemos generar algunas matrices especiales usando funciones ya incorporadas:

zeros(n) : Matriz de ceros (nxn).

```
>>Z=zeros(3); %crea una matriz de ceros de orden 3  
ans = 0 0 0  
      0 0 0  
      0 0 0
```

ones(n,m): Matriz de unos (nxm)

```
>>X=ones(3,5); %crea una matriz de unos de orden 3x5  
ans = 1 1 1 1 1  
      1 1 1 1 1  
      1 1 1 1 1
```

rand(n,m): Matriz (nxm) de números aleatorios distribuidos uniformemente entre 0 y 1

```
>> rand(3,1)  
ans = 0.2190  
      0.0470  
      0.6789
```

eye(n,m): Matriz identidad (nxm)

```
>>eye(3)  
ans= 1 0 0  
      0 1 0  
      0 0 1
```

randn(n,m): Matriz (nxm) de números aleatorios distribuidos normalmente con media cero y varianza unitaria.

```
>> randn(2)  
ans= 1.1650 0.0751  
      0.6268 0.3516
```



Notación matricial

- La mayor potencialidad de MATLAB es su capacidad para manejar matrices y vectores mediante un sistema de indexación flexible, que permite realizar numerosas operaciones con comandos sencillos que no requieren procedimientos iterativos
- MATLAB trabaja fundamentalmente con un solo tipo de elemento base: matrices
- Los escalares son matrices de un elemento por un elemento y los vectores son matrices de una fila o de una columna
- Ejemplos:
 1. Generar un vector cuyos elementos son los número del 1 al 8

```
>> v = [1:8]
```

```
v =
```

```
1      2      3      4      5      6      7      8
```



Notación matricial

1. Generar un vector cuyos elementos son los número del 1 al 8

```
>> v = [1:8]
```

```
v =
```

```
1      2      3      4      5      6      7      8
```

2. Crear un vector con una parte de v

```
>> u = v(3:6)
```

```
u =
```

```
3      4      5      6
```

3. Construir otro vector con elementos de u y v

```
>> w = [v(3:5) u(2:4)]
```

```
w =
```

```
3      4      5      4      5      6
```



Notación matricial

4. Producto escalar de vectores de tres elementos

```
>> u(1:3)*v(1:3)'  
ans =  
    26
```

5. Producto vectorial de vectores de tres elementos

```
>> m = u(1:3)'*v(1:3)  
ans =  
     3     6     9  
     4     8    12  
     5    10    15
```

6. Extraer la 2da columna de la matriz anterior

```
>> c2 = m(:,2)  
c2 =  
     6  
     8  
    10
```



Notación matricial

7. Multiplicación elemento por elemento de dos matrices

```
>> X = [1 0 1; 2 1 2; 3 2 3], Y = [3 2 1; 4 1 1; 5 2 1]
```

```
X =  
     1     0     1  
     2     1     2  
     3     2     3
```

```
Y =  
     3     2     1  
     4     1     1  
     5     2     1
```

```
>> X.*Y
```

```
ans =  
     3     0     1  
     8     1     2  
    15     4     3
```




Operaciones con matrices

- Transpuesta de una matriz

```
>> X
```

```
X =
```

```
    15     5     1
     2     6     2
     3     2     7
```

```
>> X'
```

```
ans =
```

```
    15     2     3
     5     6     2
     1     2     7
```

- Matriz inversa

```
>> inv(X)
```

```
ans =
```

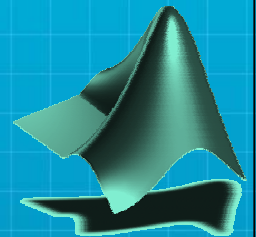
```
    0.0736   -0.0640
    0.0078
   -0.0155    0.1977    -
    0.0543
   -0.0271   -0.0291
    0.1550
```

- Determinante

```
>> det(X)
```

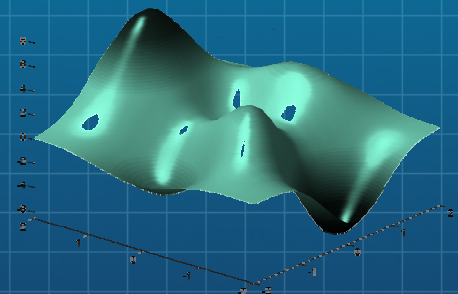
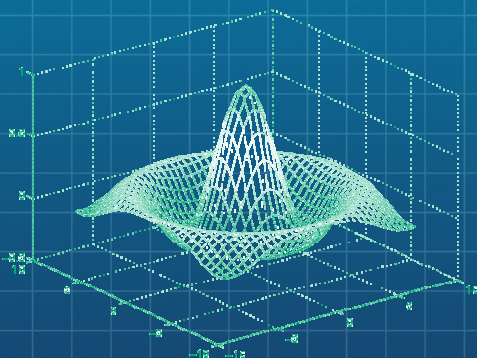
```
ans =
```

```
516
```



Vectores y Matrices

Métodos Numéricos
Aplicados a la Ingeniería





VECTORES Y MATRICES

- Las matrices son el tipo fundamental de dato en Matlab.

```
» A=[1 3 5; 6 9 2; 4 8 7]
```

A =

1	3	5
6	9	2
4	8	7

```
» det(A)
```

ans =

5

```
» A^2+3*A
```

ans =

42	79	61
86	142	68
92	164	106

MATrix LABoratory

-- datos son matrices

-- reglas del álgebra lineal



VECTORES Y MATRICES

- Vectors (arrays) are defined as

```
• >> v = [1, 2, 4, 5]
```

```
• >> w = [1; 2; 4; 5]
```

$$v = [1 \ 2 \ 4 \ 5]$$

$$w = \begin{bmatrix} 1 \\ 2 \\ 4 \\ 5 \end{bmatrix}$$

- Matrices (2D arrays) defined similarly

```
• >> A = [1,2,3;4,-5,6;5,-6,7]
```

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & -5 & 6 \\ 5 & -6 & 7 \end{bmatrix}$$

Ejercicio

size(c) = [fila,columna] length(c) = max(size(c))

Command Window

```
>> a = [1,2,3,4,5]
```

```
a =
```

```
1 2 3 4 5
```

```
>> a = [1;2;3;4;5];
```

```
>>
```

```
>> b = [1 2 3;4 5 6];
```

```
>> length(a);
```

```
>> length(a)
```

```
ans =
```

```
5
```

```
>> c = [1 2;3 4;5 6];
```

```
>> size(a)
```

```
ans =
```

```
5 1
```

```
>> size(c)
```

```
ans =
```

```
3 2
```

Array Editor - a

	1	2	3	4	5
1	1	2	3	4	5

size(a) = [1 5] length(a) = 5

Array Editor

	1
1	1
2	2
3	3
4	4
5	5

size(a) = [5 1] length(a) = 5

Array Editor - b

	1	2	3
1	1	2	3
2	4	5	6

size(b) = [2 3] length(b) = 3

Array Editor - c

	1	2
1	1	2
2	3	4
3	5	6

size(c) = [3 2] length(c) = 3

Array Editor: Ventana donde podemos observar las matrices

Matrix Multiplication

- Example

$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 2 & 1 \end{bmatrix} \begin{matrix} \text{3 columns} \end{matrix} \begin{bmatrix} 2 & 3 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 2 \end{bmatrix} \begin{matrix} \text{3 rows} \end{matrix} = \begin{bmatrix} 3 & 2 & -1 \\ 1 & 1 & 0 \end{bmatrix}$$

```
>> [1,0,-1;0,2,1] * [2,3,1;1,0,-1;-1,1,2]
```

```
ans =
```

```
3 2 -1
1 1 0
```



Vectores y matrices

Definición de vectores:

- **Vectores fila**; elementos separados por blancos o comas
`>> v =[2 3 4]`
- **Vectores columna**: elementos separados por **punto y coma (;)**
`>> w =[2;3;4;7;9;8]`
- Dimensión de un vector w: **length(w)**
- Generación de vectores
 - Especificando el incremento **h** de sus componentes **v=a:h:b**
 - Especificando su dimensión **n**: **linspace(a,b,n)** (por defecto n=100)
 - Componentes logarítmicamente espaciadas **logspace(a,b,n)** (n puntos logarítmicamente espaciados entre 10^a y 10^b . Por defecto n=50)



Vectores y matrices

Definición de matrices:

- No hace falta establecer de antemano su tamaño (se puede definir un tamaño y cambiarlo posteriormente).
- **Las matrices se definen por filas**; los elementos de una misma fila están separados por blancos o comas. Las filas están separadas por punto y coma (;).
» `M=[3 4 5; 6 7 8; 1 -1 0]`
- **Matriz vacía**: `M=[];`
- Información de un elemento: `M(1,3)`, de una fila `M(2,:)`, de una columna `M(:,3)`.
- Cambiar el valor de algún elemento: `M(2,3)=1;`
- Eliminar una columna: `M(:,1)=[ ]`, una fila: `M(2,:)=[];`



Vectores y matrices

Definición de matrices:

- Generación de matrices:
 - Generación de una matriz de ceros, **zeros(n,m)**
 - Generación de una matriz de unos, **ones(n,m)**
 - Inicialización de una matriz identidad **eye(n,m)**
 - Generación de una matriz de elementos aleatorios **rand(n,m)**
- Añadir matrices: [X Y] columnas, [X; Y] filas



Vectores y matrices

Operaciones con Matrices y vectores:

- Para definir matrices se utiliza:
 - []** constructor
 - ,** separador de columnas
 - ;** separador de filas
- En lugar de coma (,) puede utilizarse un espacio, y en lugar de punto y coma (;) puede utilizarse un retorno de carro
- Ejemplo:

A=[1,2,3; 4,5,6]

o simplemente:

A=[1 2 3

4 5 6]



Operaciones con vectores y matrices

Operaciones de vectores y matrices con escalares:

v: vector, **k:** escalar:

- $v+k$ adición o suma
- $v-k$ sustracción o resta
- $v*k$ multiplicación
- v/k divide cada elemento de v por k
- $k./v$ divide k por cada elemento de v
- $v.^k$ potenciación de cada componente de v a k
- $k.^v$ potenciación k elevado a cada componente de v



Operaciones con vectores y matrices

Operaciones con vectores y matrices:

- $+$ adición o suma
- $-$ sustracción o resta
- $*$ multiplicación matricial
- $.*$ producto elemento a elemento
- $^$ potenciación
- $.^$ elevar a una potencia elemento a elemento
- $\backslash$ división-izquierda
- $/$ división-derecha
- $./$ y $.\backslash$ división elemento a elemento
- matriz traspuesta: $B=A'$ (en complejos calcula la traspuesta conjugada, sólo la traspuesta es $B=A.'$)



Funciones para vectores y matrices

Funciones de MATLAB para vectores y matrices:

- **sum(v)** suma los elementos de un vector
- **prod(v)** producto de los elementos de un vector
- **dot(v,w)** producto escalar de vectores
- **cross(v,w)** producto vectorial de vectores
- **mean(v)** (hace la media)
- **diff(v)** (vector cuyos elementos son la resta de los elemento de v)
- **[y,k] = max(v)** valor máximo de las componentes de un vector (k indica posición).
min(v) (valor mínimo).

El valor máximo de una matriz M se obtendría como **max(max(M))** y el mínimo **min(min(v))**



Funciones para vectores y matrices

Funciones de Matlab para vectores y matrices

- **[n,m]=size(M)** te da el número de filas y columnas
- matriz inversa: **B=inv(M)**, rango: **rank(M)**
- **diag(M)**: Obtención de la diagonal de una matriz.
sum(diag(M)) calcula la traza de la matriz A.
diag(M,k) busca la k-ésima diagonal.
- **norm(M)** norma de una matriz (máximo de valores absolutos de los elementos de A)
- **flipud(M)** reordena la matriz, haciendo la simétrica respecto de eje horizontal.
fliplr(M) reordena la matriz, haciendo la simétrica respecto de un eje vertical
- **[V, landa]=eig(M)** da una matriz diagonal **landa** con los autovalores y otra **V** cuyas columnas son los autovectores de M



Operaciones con Matrices y vectores:

- Pruebe los siguientes comandos

$A = [1, 2, 3; 4, 5, 6]$

$B = A'$

$C = A * B$

$D = B * A$

$v = [1, 2, 3, 4]$

$w = [5; 6; 7; 8]$

$x = v * w$

$Y = w * v$

$M = [1:10; 11:20; 21:30]$

$V = [1:0.3:10]$



Operaciones con Matrices y vectores:

- Pruebe los siguientes comandos

$A = [1, 2, 3; 4, 5, 6]$ Define la matriz A de 2x3

$B = A'$ Define B como la matriz transpuesta de A

$C = A * B$ C es la multiplicación de matrices $A * B$

$D = B * A$ D es la multiplicación de matrices $B * A$

$v = [1, 2, 3, 4]$ Define el vector fila v

$w = [5; 6; 7; 8]$ Define el vector columna w

$x = v * w$ x es la multiplicación de matrices $v * w$

$Y = w * v$ Y es la multiplicación de matrices $w * v$

$M = [1:10; 11:20; 21:30]$ matriz de 3x10, elementos 1 al 30

$V = [1:0.3:10]$ vector con elementos del 1 al 10 cuyas componentes se forman sumando 0.3 a la anterior



Funciones que devuelven matrices

- `eye(4)` Forma la matriz unidad de 4x4
- `zeros(3,5)` Forma una matriz de ceros de 3x5
- `ones(3)` Forma una matriz de unos de 3x3
- `ones(2,4)` Idem de tamaño 2x4
- `rand(3)` forma una matriz de 3x3 de números aleatorios entre 0 y 1, con distribución uniforme
- `rand(4,2)` Idem de tamaño 4x2
- Probar los siguientes comandos:

`A= magic(3)`

`v=[1:10]`

`size(A)`

`size(v)`

`length(A)`

`length(v)`

`sum(A)`

`sum(v)`



Operadores `.*` `./` `.^`

- En MATLAB se puede aplicar elemento a elemento los operadores (`*` `/` `^`) Para ello se los precede por un punto(`.`)

» `[1 2 3 4]^2`

??? Error using ==> `^`

Matrix must be square.

» `[1 2 3 4].^2`

ans =

1 4 9 16

» `[1 2 3 4]*[1 -1 1 -1]`

??? Error using ==> `*`

Inner matrix dimensions must agree.

» `[1 2 3 4].*[1 -1 1 -1]`

ans =

1 -2 3 -4



Operadores relacionales

<	menor que
>	mayor que
<=	menor o igual que
>=	mayor o igual que
==	igual que
~=	distinto que

Si una comparación se cumple el resultado es 1 (true), mientras que si no se cumple es 0 (false).

- Operadores lógicos

&	and
	or
~	negación lógica



- Cuando los operadores relacionales se aplican a matrices del mismo tamaño, la comparación se realiza elemento a elemento

```
>> A=[1,2;3,4];
```

```
>> B=[4,3;3,2];
```

```
>> A==B
```

```
ans =
```

```
0    0
```

```
1    0
```

```
>> A~=B
```

```
ans =
```

```
1    1
```

```
0    1
```



- Si se compara una matriz con un escalar, La comparación se realiza entre el escalar y cada elemento de la matriz.

```
>> A=[1,2,2;2,3,3;4,4,2];
```

```
>> A==2
```

```
ans =
```

```
0    1    1
```

```
1    0    0
```

```
0    0    1
```



- Creando matrices con submatrices
 - Se puede crear una nueva matriz componiendo como submatrices, matrices definidas previamente. A modo de ejemplo ejecútense las siguientes instrucciones:

```
>> A=zeros(2);
```

```
>> B=ones(2,3);
```

```
>> C=[A,B;1:5]
```

```
C =
```

```
0    0    1    1    1
```

```
0    0    1    1    1
```

```
1    2    3    4    5
```




Numerical Linear Algebra

- Basic numerical linear algebra
- `>> z=[1;2;3]; x=inv(A)*z`
- `>> x=A\z`

$$Ax = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$
$$x = A^{-1} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

- Many standard functions predefined
- `>> det(A)`
- `>> rank(A)`
- `>> eig(A)`
- The number of input/output arguments can often be varied
- `>> [V,D]=eig(A)`



Vectores

- Edición

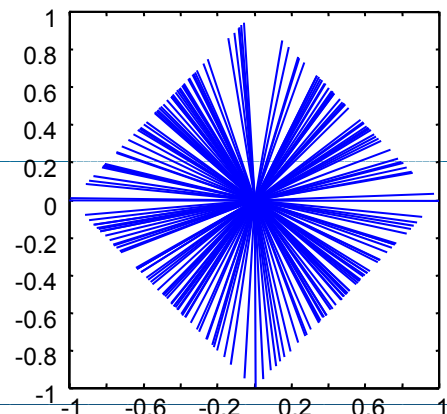
- » `u = [1 2 3]`
- » `v = [1,2,3]`
- » `w = [1;2;3]`
- » `w = [1
2
3]`

- Progresivos

- » `0:0.1:10`
- » `linspace(0,1,11)`

- Normas

- » `norm(v,2)`
- » `norm(v,1)`
- » `norm(v,inf)`





Operaciones con vectores

- Suma: $u+v$
 - de comps.: `sum(u)`
- Productos
 - por escalar: $2*u$
 - escalar: `dot(u,v)`
 - elemental: `u.*v`
 - matricial: `u*w, w*u`
 - de comps.: `prod(u)`
- Transpuesta: u'
- Voltar
 - » `flip1r(x)`
 - » `flipud(x')`
- Funciones
 - » `x = -1:0.01:1`
 - » `y = tanh(x)`
 - » `plot(x,y)`



Matrices

- Edición
 - » `A = [1,2;3,4]`
 - » `B = [-1 -2
-3 -4]`
- Elemento: `A(2,1)`
- Fila: `A(2,:)`
- Columna: `A(:,1)`
- Bloques
 - » `M = [A,B;B,A]`
- Submatrices
 - » `M41 = M(1:3,2:4)`
 - » `fil = [1,2,4]`
 - » `col = [1,3,4]`
 - » `M32 = M(fil,col)`



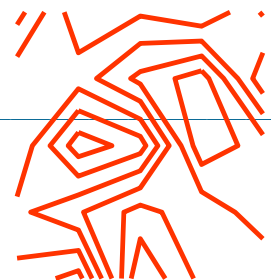
Operaciones con matrices

- | | | |
|------------------|----------------------------|------------------|
| • Suma y resta: | $+$ $-$ | • Determinante |
| • Producto: | $*$ $.*$ | » det(A) |
| • Potencia: | $^{\wedge}$ $.^{\wedge}$ | • Inversa |
| • Cociente izq.: | $/$ $./$ | » inv(A) |
| • Cociente der.: | $\backslash$ $.\backslash$ | • Rango |
| • Transpuesta: | $'$ $.'$ | » rank(A) |



Matrices usuales

- | | |
|----------------------------|-------------------|
| • Identidad de orden n: | eye(n) |
| • Nula de tamaño m×n: | zeros(m,n) |
| • Matriz de unos: | ones(m,n) |
| • Matriz aleatoria: | rand(m,n) |
| • Matriz de Hilbert: | hilb(n) |
| • Matriz de van der Monde: | vander(x) |





Ejercicio

$$a(1,1) = 7.9787$$

$$a(:,1) = \begin{matrix} 7.9787 \\ 11.229 \\ 11.015 \\ 13.385 \\ 11.183 \end{matrix}$$

	1	2	3	4	5
1	7.9787	8.7128	10	11.791	11.138
2	11.229	10.761	9.3643	11.462	9.4887
3	11.015	7.9818	12.19	11.156	9.2451
4	13.385	9.961	6.252	10.081	9.4082
5	11.183	9.9036	10.856	11.354	7.0497



Ejercicio

$$a(1:2:end,1) = \begin{matrix} 7.9787 \\ 11.015 \\ 11.183 \end{matrix}$$

	1	2	3	4	5
1	7.9787	8.7128	10	11.791	11.138
2	11.229	10.761	9.3643	11.462	9.4887
3	11.015	7.9818	12.19	11.156	9.2451
4	13.385	9.961	6.252	10.081	9.4082
5	11.183	9.9036	10.856	11.354	7.0497

	1	2	3	4	5
1	7.9787	8.7128	10	11.791	11.138
2	11.229	10.761	9.3643	11.462	9.4887
3	11.015	7.9818	12.19	11.156	9.2451
4	13.385	9.961	6.252	10.081	9.4082
5	11.183	9.9036	10.856	11.354	7.0497

$$a(3,:) = 11.015 \ 7.9818 \ 12.19 \ 11.156 \ 9.2451$$



Ejercicio

	1	2	3	4	5
1	7.9787	8.7128	10	11.791	11.138
2	11.229	10.761	9.3643	11.462	9.4887
3	11.015	7.9818	12.19	11.156	9.2451
4	13.385	9.961	6.252	10.081	9.4082
5	11.183	9.9036	10.856	11.354	7.0497

$a(3,1:3) = 11.015 \ 7.9818 \ 12.19$



Elementos básicos en MATLAB

- Redondeo:
- $a = 4.566$
- $b = \text{fix}(a)$ valor entero = 4
- $c = \text{roundn}(a, -2)$, redondeo al decimal mas próximo 4.57
- $d = \text{ceil}(a)$ redondeo al entero próximo = 5
- Estadística Simple
- $a = 100 * \text{rand}(15, 1);$
- $[oa, pa] = \text{sort}(a);$ Orden ascendente
- $[od, pd] = \text{sort}(a, 'descend');$ Orden descendente
- $p = \text{hist}(a, 5);$ Histograma
- $y = \text{prctile}(a, [25 \ 50 \ 75]);$ Percetiles



Ejercicio

	1
1	64.053
2	20.907
3	37.982
4	78.333
5	68.085
6	46.11
7	56.783
8	79.421
9	5.9183
10	60.287
11	5.0269
12	41.537
13	30.5
14	87.437
15	1.5009

a

	1
1	1.5009
2	5.0269
3	5.9183
4	20.907
5	30.5
6	37.982
7	41.537
8	46.11
9	56.783
10	60.287
11	64.053
12	68.085
13	78.333
14	79.421
15	87.437

Orden
ascendente

	1
1	15
2	11
3	9
4	2
5	13
6	3
7	12
8	6
9	7
10	10
11	1
12	5
13	4
14	8
15	14

Posición Orden
ascendente

	1
1	87.437
2	79.421
3	78.333
4	68.085
5	64.053
6	60.287
7	56.783
8	46.11
9	41.537
10	37.982
11	30.5
12	20.907
13	5.9183
14	5.0269
15	1.5009

Orden
descendente

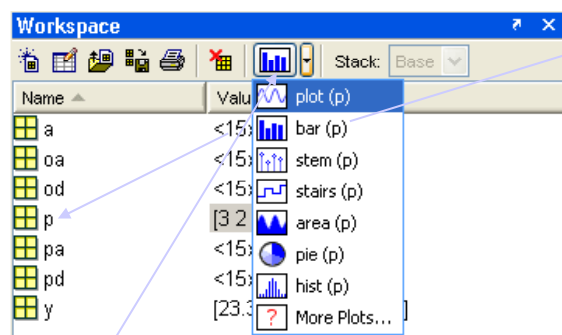
	1
1	14
2	8
3	4
4	5
5	1
6	10
7	7
8	6
9	12
10	3
11	13
12	2
13	9
14	11
15	15

Posición Orden
descendente

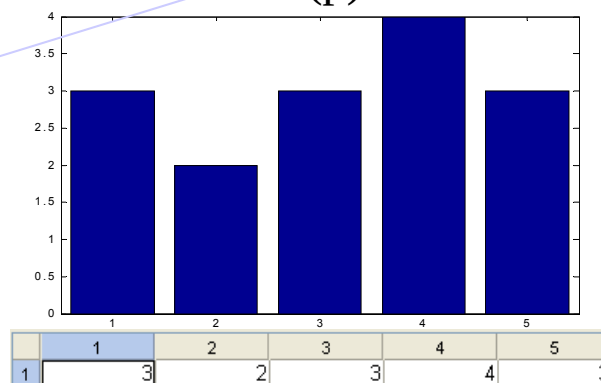


Ejercicio

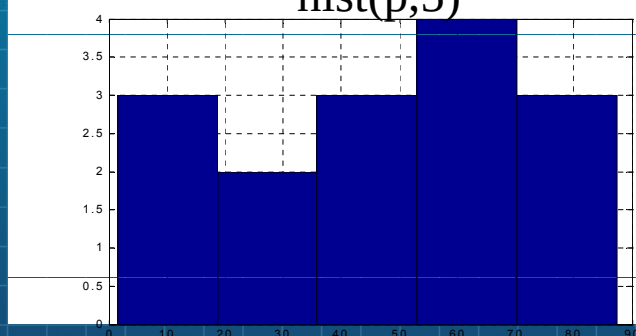
Opciones para generar gráficas



bar(p)



hist(p,5)



Percentiles

	1	2	3
1	23.3052	46.11	67.077

25% 50% 75%



Ejercicios

- Resolviendo sistemas de ecuaciones lineales

$$1.2 x_1 + 2.2 x_2 = 21.4$$

$$7.1 x_1 + 0.5 x_2 = 39$$

$$\begin{bmatrix} 1.2 & 2.2 \\ 7.1 & 0.5 \end{bmatrix} \times \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 21.4 \\ 39 \end{bmatrix}$$

$$A = [1.2, 2.2 ; 7.1, 0.5]$$

$$R = [21.4 ; 39]$$

$$X = \text{inv}(A) * R$$



Ejercicios:

- 1) Obtener el producto interior $a \cdot b$

$$a = [1, 2, 3, 4]$$

$$b = [5, 6, 7, 8]$$

- 2) Verificar si efectivamente `magic(4)` es un cuadro mágico comprobando que las filas, columnas y diagonales principales sumen exactamente el mismo valor (utilizar funciones *diag* y *rot90*)



Solución de los Ejercicios

- 1) Obtener el producto interior $a \cdot b$

$a=[1,2,3,4]$

$b=[5,6,7,8]$

Rta: $a \cdot b'$

- 2) Verificar si efectivamente `magic(4)` es un cuadro mágico comprobando que las filas, columnas y diagonales principales sumen exactamente el mismo valor (utilizar funciones *diag* y *rot90*)

Rta: hay que realizar las siguientes operaciones:

$A=\text{magic}(4)$

$\text{sum}(A)$ se obtienen las sumas de las columnas

$\text{sum}(A')$ se obtienen las sumas de las filas

$\text{sum}(\text{diag}(A))$ se obtiene la suma de la diagonal principal

$\text{sum}(\text{diag}(\text{rot90}(A)))$ se obtiene la suma de la otra diagonal



Ejercicios

- 1) Escribir una expresión que compute la cantidad de números 3 que posee una matriz A .
- 2) Escriba una expresión que devuelva el valor máximo de cada fila de una matriz A (función `max`).
- 3) Escriba una expresión que devuelva `true` (1) si todos los elementos de una matriz son iguales.



Solución de los Ejercicios

- 1) Escribir una expresión que compute la cantidad de numeros 3 que posee una matriz A.

`sum(sum(A==3))`

- 2) Escriba una expresión que devuelva el valor máximo de cada fila de una matriz A (función max).

`max(A)'`

- 3) Escriba una expresión que devuelva true (1) si todos los elementos de una matriz A son iguales.

`max(max(A))==min(min(A))`



- Acceso a los elementos de una matriz

- Los elementos de las matrices se acceden por sus dos índices. Por ejemplo A(1,2) ó A(i,j).
- Sin embargo las matrices se almacenan por columnas y teniendo en cuenta esto puede accederse a cualquier elemento con un sólo subíndice. Por ejemplo

```
>> A=[1,2,3;4,5,6;7,8,9];
```

```
>> A(4)
```

```
ans =
```

```
2
```

- Probar qué devuelve A(:)

- Rta: el vector columna formado por la concatenación de las columnas de A



- Operador (:) como índice

```
>> A=magic(4)
```

```
A =
```

16	2	3	13
5	11	10	8
9	7	6	12
4	14	15	1



- Operador (:) como índice

```
>> A=magic(4)
```

```
A =
```

16	2	3	13
5	11	10	8
9	7	6	12
4	14	15	1

```
>> A(2:3,2:4)
```

```
ans =
```

11	10	8
7	6	12



- Operador (:) como índice

```
>> A=magic(4)
```

```
A =
```

```
16    2    3   13
```

```
5    11   10    8
```

```
9     7    6   12
```

```
4    14   15    1
```

```
>> A(2,:)
```

```
ans =
```

```
5    11   10    8
```



- Operador (:) como índice

```
>> A=magic(4)
```

```
A =
```

```
16    2    3   13
```

```
5    11   10    8
```

```
9     7    6   12
```

```
4    14   15    1
```

```
>> A(:,1)
```

```
ans =
```

```
16
```

```
5
```

```
9
```

```
4
```

- Operador (:) como índice

```
>> A=magic(4)
```

```
A =
```

16	2	3	13
5	11	10	8
9	7	6	12
4	14	15	1

```
>> A(4:-1:1,:)
```

4	14	15	1
9	7	6	12
5	11	10	8
16	2	3	13

- Operador (:) como índice

```
>> A=magic(4)
```

```
A =
```

16	2	3	13
5	11	10	8
9	7	6	12
4	14	15	1

```
>> A([1,3],:)
```

16	2	3	13
9	7	6	12

Obsérvese que también puede utilizarse un vector como índice. En este caso sirve para poder seleccionar filas disjuntas



Ejercicios

- 1) Construir una matriz A de 2x6 formada por la segunda y tercera fila de `magic(6)`
- 2) Eliminar la columna 3
- 3) Obtener el vector suma por columna y agregarlo como última fila de A
- 4) Obtener el vector suma por fila y agregarlo como última columna de A

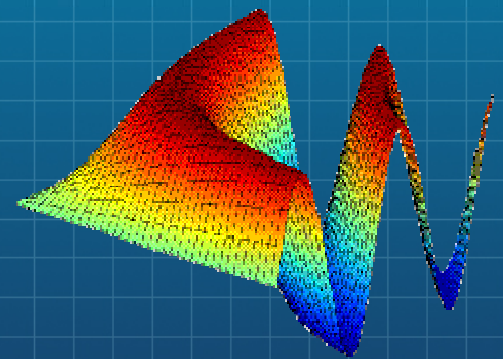
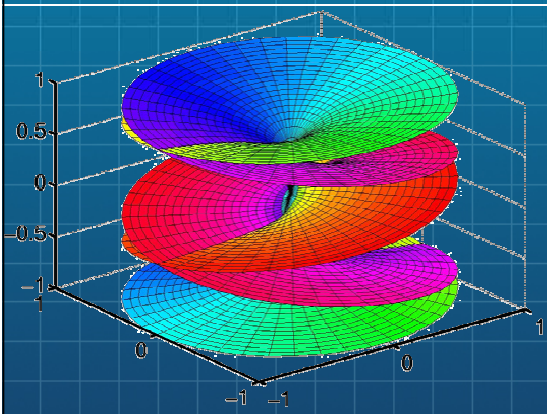


Solución de los Ejercicios

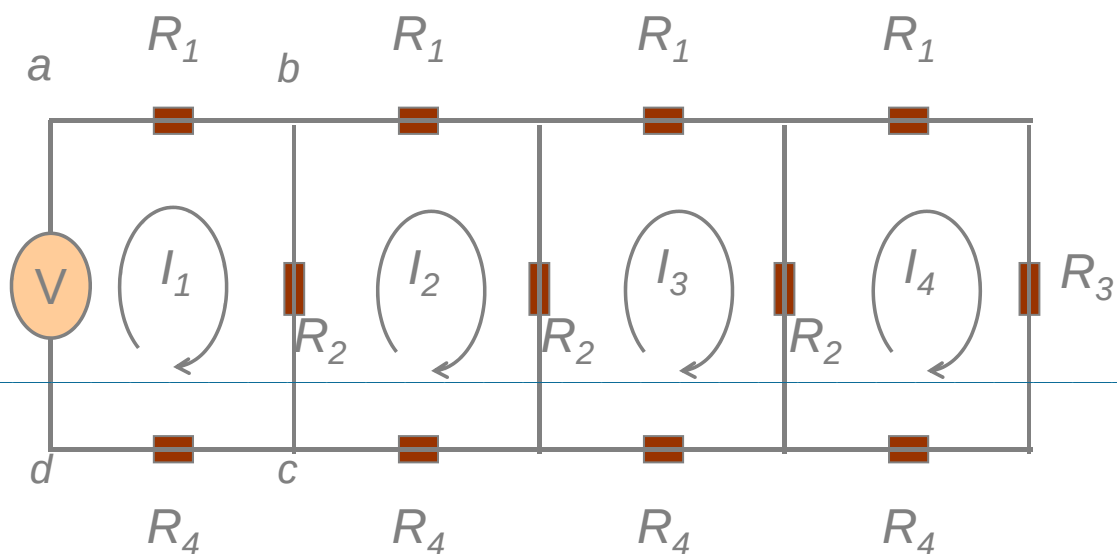
- 1) Construir una matriz A de 2x6 formada por la segunda y tercera fila de `magic(6)`
`A=magic(6);`
`A=A(2:3,:)`
- 2) Eliminar la columna 3
`A=A(:,[1,2,4:6])`
- 3) Obtener el vector suma por columna y agregarlo como última fila de A
`A=[A;sum(A)]`
- 4) Obtener el vector suma por fila y agregarlo como última columna de A
`A=[A,sum(A)']`

Aplicaciones a la Ingeniería

Métodos Numéricos Aplicados a la Ingeniería

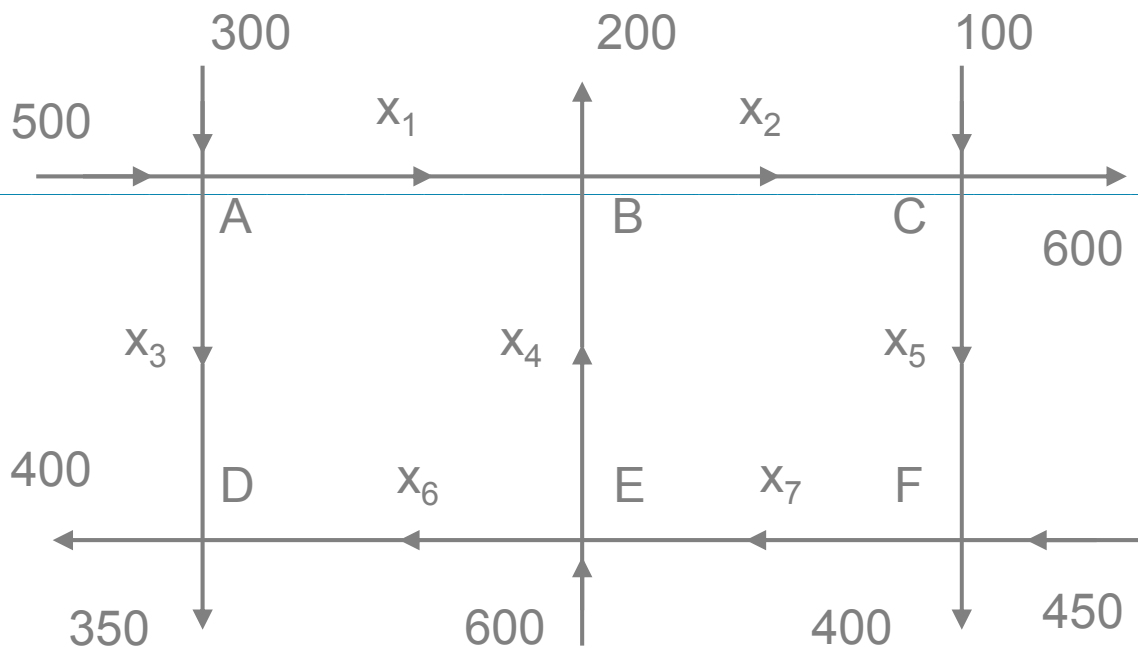


Una red eléctrica





Una red de calles

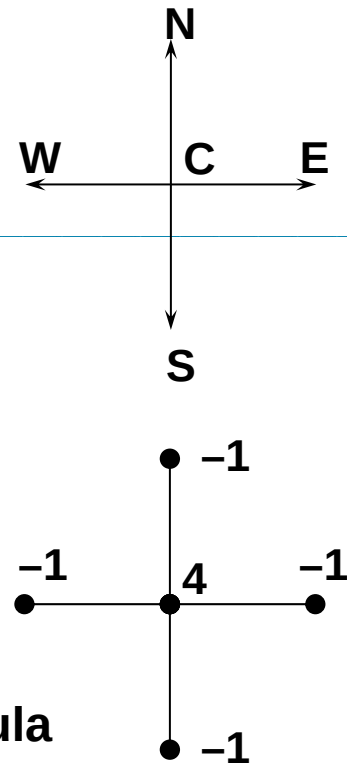
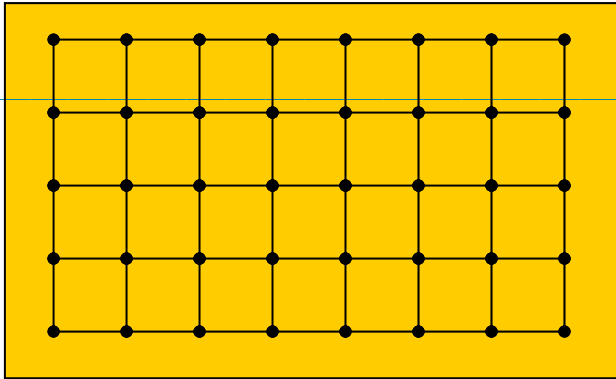


Matriz de incidencia

		Calle						
		1	2	3	4	5	6	7
C r u c e	A	1	0	1	0	0	0	0
	B	-1	1	0	-1	0	0	0
	C	0	-1	0	0	1	0	0
	D	0	0	-1	0	0	-1	0
	E	0	0	0	1	0	1	-1
	F	0	0	0	0	-1	0	1



Ecuación del Calor en un rectángulo



- $T_C = (T_W + T_N + T_S + T_E)/4$

Molécula



Ecuación del Calor

- Modelo matemático

$$T_1 = (T_0 + T_2)/2$$

$$T_2 = (T_1 + T_3)/2$$

$$T_3 = (T_2 + T_4)/2$$

⋮

$$T_n = (T_{n-1} + T_{n+1})/2$$

- Matriz asociada

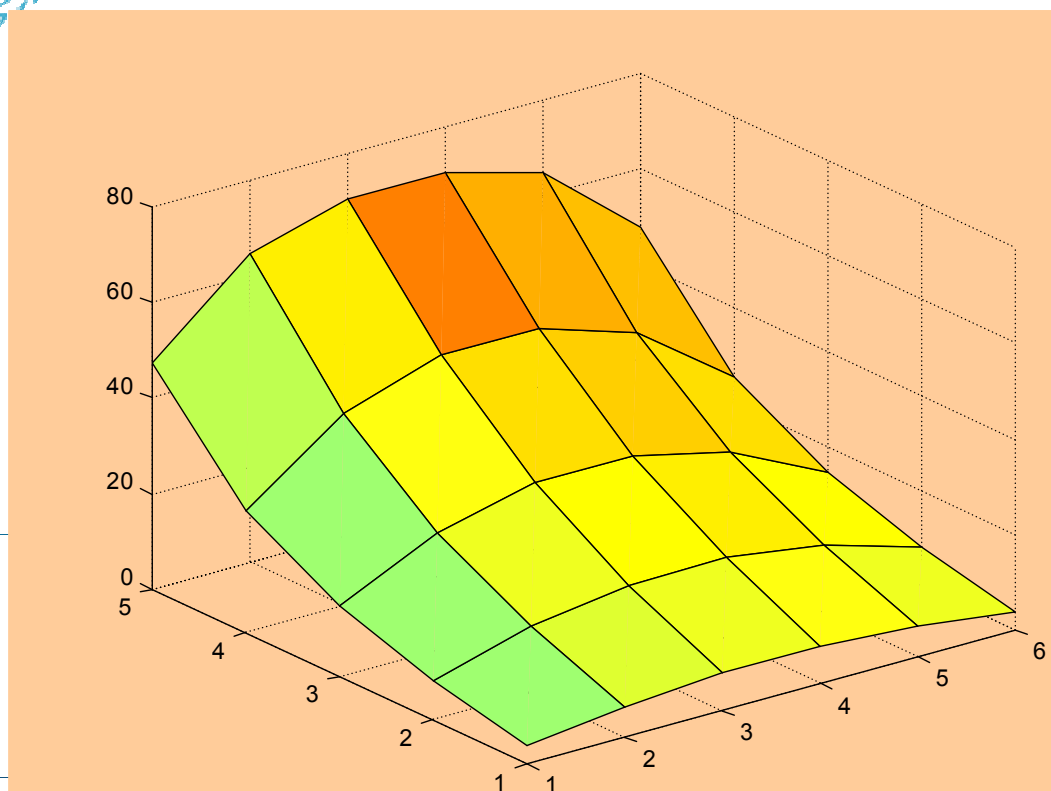
$$\begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & -1 & 2 & \ddots & \\ & & \ddots & \ddots & -1 \\ & & & -1 & 2 \end{pmatrix}$$



Generación de la matriz con MATLAB

```
function A = calor2D(n,m)
p = n*m;
v = ones(1,p-1);
for k=n:n:p-1, v(k) = 0; end
w = ones(1,p-n);
A = 4*eye(p)...
    - diag(v,1) - diag(v,-1)...
    - diag(w,n) - diag(w,-n);
```

Gráfica del problema de transferencia de calor





Aplicaciones a la Ingeniería

La utilización de los Métodos Iterativos para la resolución de sistemas de ecuaciones lineales es aconsejable cuando se abordan problemas de gran dimensión y dispersos.

Existen multitud de aplicaciones donde la matriz de coeficientes es dispersa y de gran dimensión

Ejemplos:

- ▢ Ecuaciones en derivadas parciales
- ▢ Problemas de valores en la frontera
- ▢ Aproximación de funciones (Splines)



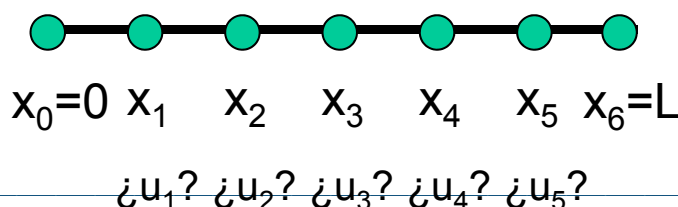
Ecuación de transmisión del calor

La ecuación de transmisión del calor en una barra en régimen estacionario, viene dada por la siguiente ecuación diferencial.

$$\frac{d^2 u}{dx^2} = 0 \quad 0 \leq x \leq L$$

con las condiciones de frontera $u(0)=0$ y $u(L)=1$.

$$u_0 = u(0) = 0 \qquad h \qquad 1 = u(L) = u_6$$



$$L=6$$
$$h=L/6=1$$



Ecuación de transmisión del calor (2)

Aproximando mediante diferencias finitas

$$\frac{d^2 u}{dx^2} \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2}$$

y sustituyendo en la ecuación original, queda:

$$\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} = 0 \quad i = 1, \dots, 5$$

A partir de estas 5 ecuaciones se obtiene un sistema tridiagonal

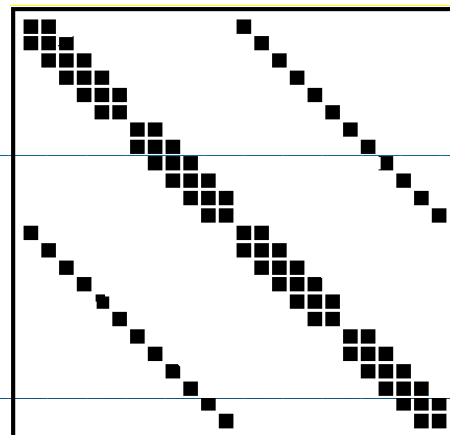
$$\begin{pmatrix} -2 & 1 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 1 & -2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ -1 \end{pmatrix}$$

Ecuación de Poisson

Como resultado de la discretización de la ecuación de Poisson.

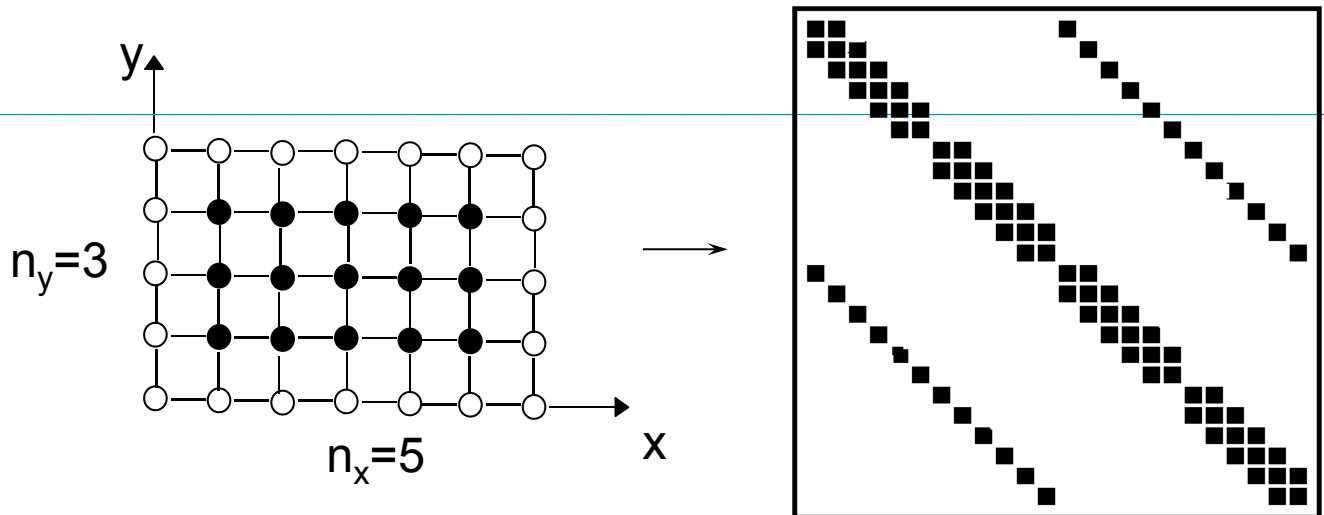
$$-\left(\frac{\partial^2 u}{x^2} + \frac{\partial^2 u}{y^2} \right) = f(x, y) \quad \text{en } \Omega \quad \text{y} \quad u = 0 \quad \text{en } \Gamma$$

Aparece un S.E.L. con la matriz de coeficientes dispersa de la forma:



Introducción. Ecuación de Poisson

Si llamamos Ω a la superficie de puntos negros y Γ a contorno de punto blancos.



La matriz resultante es de dimensión $n_x n_y = 15$.

CASOS DE APLICACIÓN

Problema Académico

SISTEMAS DE ECUACIONES LINEALES

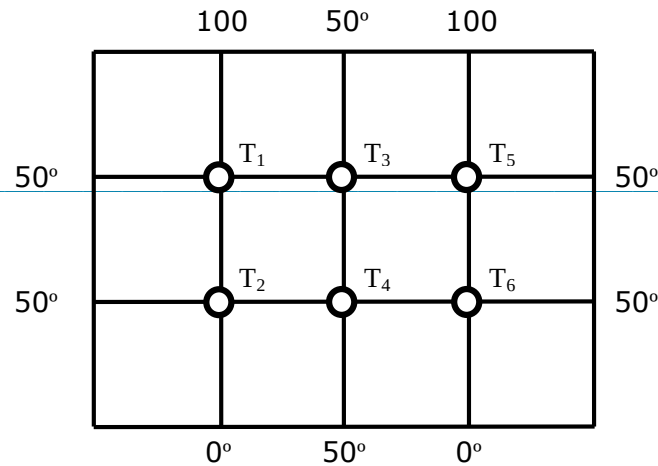
El diagrama adjunto representa la discretización del problema del calor en una placa. Se trata de determinar la temperatura en los nodos interiores de la malla T_j , $j = 1, 2, \dots, 6$, conocidas las temperaturas en el borde y suponiendo que hay equilibrio térmico, es decir que las temperaturas no varían.

En el modelo discreto se supone que, en el equilibrio, la temperatura en cada nodo es la media de las temperaturas en los nodos vecinos.

- Obtener el sistema lineal correspondiente a los datos de la figura, formulando las condiciones que han de verificar las temperaturas de los nodos interiores.
- Resolver el sistema por un método directo.
- Iterar por el método de Jacobi hasta que la variación máxima de la temperatura en un nodo sea inferior a 0.01. ¿Cuál es la máxima desviación con respecto a la solución obtenida en el primer apartado?
- Iterar por el método de Gauss-Seidel y comparar los resultados con los del método de Jacobi.



CASOS DE APLICACIÓN



NOTA.- suponga que la temperatura en cada nodo T_{ij} es la media de las temperaturas de los 4 nodos vecinos:

$$T_{ij} = \frac{1}{4}(T_{i-1,j} + T_{i,j-1} + T_{i,j+1} + T_{i+1,j})$$



Las condiciones de equilibrio de cada nodo, proporcionan el sistema ya preparado para aplicar un método iterativo

$$T_1 = \frac{1}{4}(50 + 100 + T_2 + T_3)$$

$$T_2 = \frac{1}{4}(50 + T_1 + 0 + T_4)$$

$$T_3 = \frac{1}{4}(T_1 + 50 + T_4 + T_5)$$

$$T_4 = \frac{1}{4}(T_2 + T_3 + 50 + T_6)$$

$$T_5 = \frac{1}{4}(T_3 + 100 + T_6 + 50)$$

$$T_6 = \frac{1}{4}(T_4 + T_5 + 0 + 50)$$

Escribimos el sistema en forma normal y lo resolvemos con MATLAB

$$4T_1 - T_2 - T_3 = 150$$

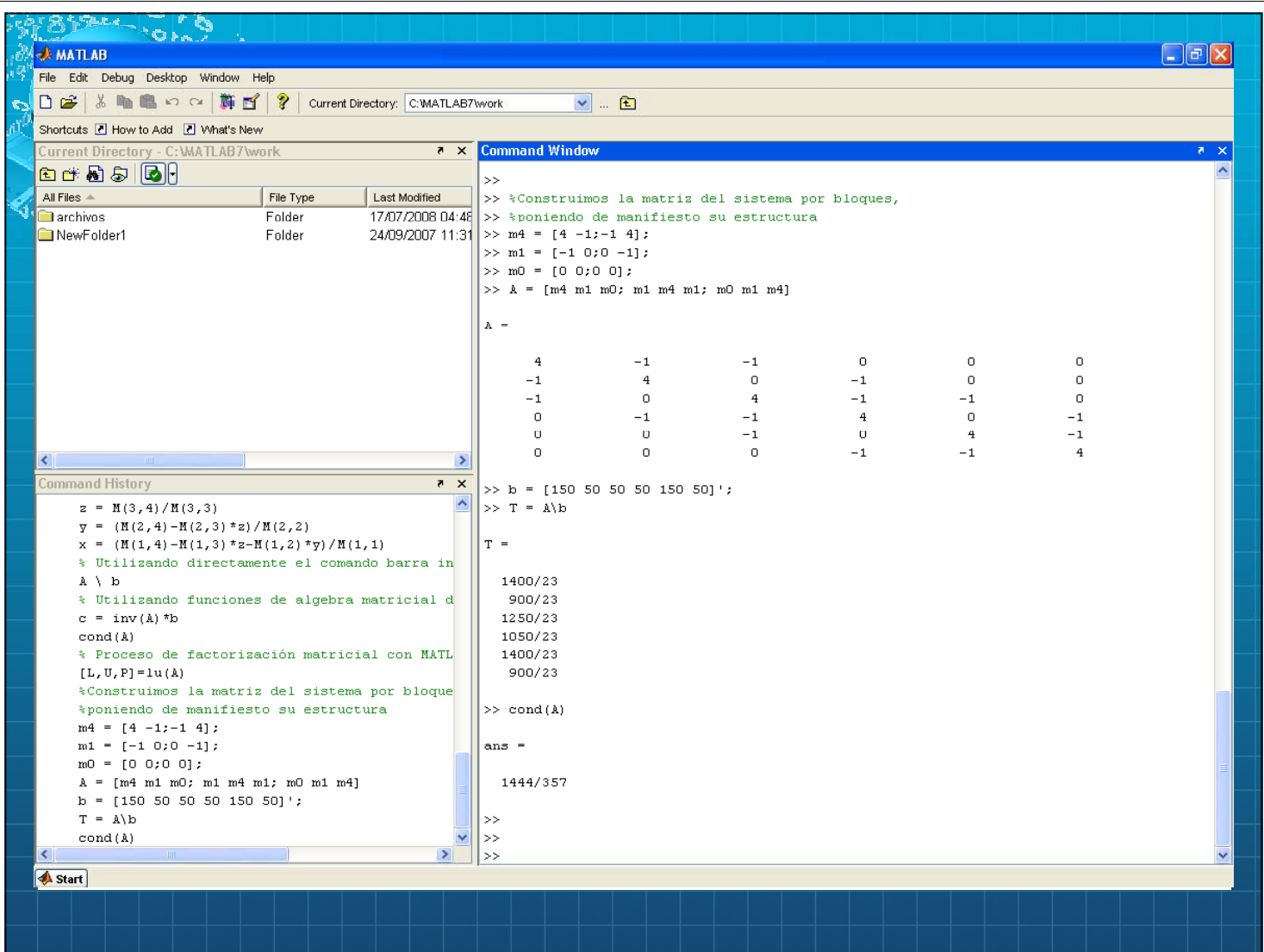
$$-T_1 + 4T_2 - T_4 = 50$$

$$-T_1 + 4T_3 - T_4 - T_5 = 50$$

$$-T_2 - T_3 + 4T_4 - T_6 = 50$$

$$-T_3 + 4T_5 - T_6 = 150$$

$$-T_4 - T_5 + 4T_6 = 50$$



Comentario

Sea Ω_i la temperatura en el nodo (i) . La ecuación que rige la distribución de temperatura en una placa es:

$$\left(\frac{\partial^2 \Omega}{\partial x^2} + \frac{\partial^2 \Omega}{\partial y^2} \right) = \frac{Q}{kt}$$

- Ω : Variable dependiente, que representa la temperatura del elemento
- t : Grosor de la placa
- k : Conductividad
- Q : Razón de pérdida de calor por unidad de volumen

La ecuación que rige esta situación es con $Q = 0$, donde estas aproximaciones se usan para las derivadas de segundo orden en un nodo central donde la temperatura es ΩO .

RAZÓN DE FLUJO
HACIA DENTRO

RAZÓN DE FLUJO
HACIA FUERA

RAZÓN DE PÉRDIDA DE CALOR
DESDE LA SUPERFICIE

Nota.- Los matemáticos cuentan con una denominación y un símbolo especial para indicar la suma de las segundas derivadas parciales. Se denomina LAPLACIANO y se representa por el símbolo $\nabla^2 \Omega$. Por tanto, suele representarse como:

$$\nabla^2 \Omega = \frac{Q}{kt}$$

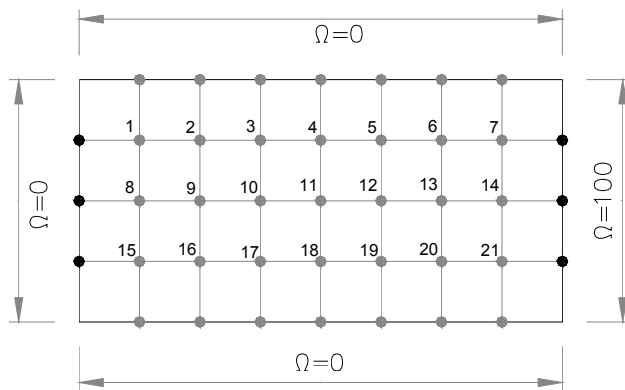
Comentario

Para el problema señalado, hay un dispositivo que puede usarse para recordar esta aproximación al Laplaciano, que se denomina operador gráfico:

$$\nabla^2 \Omega = \frac{1}{h^2} \begin{Bmatrix} 1 \\ 1 & -4 & 1 \\ 1 \\ 1 \end{Bmatrix} \Omega O \equiv \nabla^2 \Omega = \frac{(\Omega_L + \Omega_R + \Omega_A + \Omega_B - 4\Omega O)}{h^2}$$

Mediante el Método de LIEBMANN, (con condiciones en la frontera de DIRICHLET):

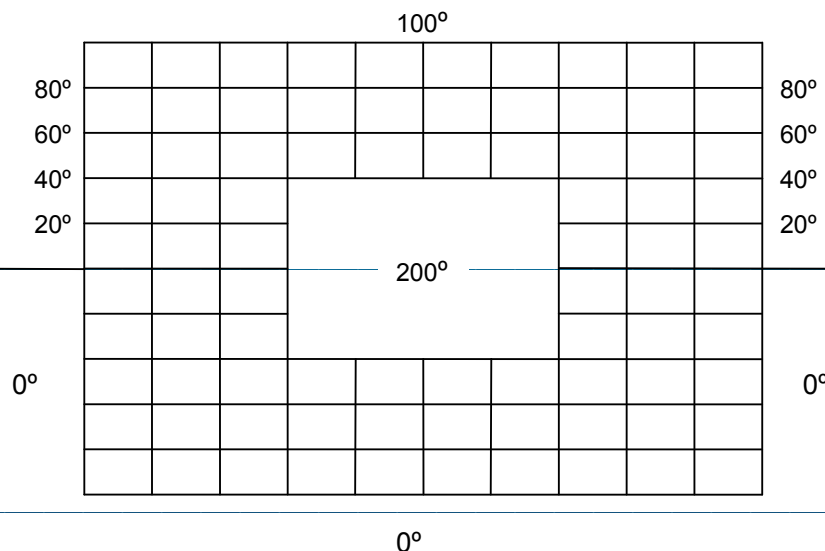
$$\Omega O = \frac{1}{4} \begin{Bmatrix} 1 \\ 1 & 0 & 1 \\ 1 \\ 1 \end{Bmatrix} \equiv \Omega_{i,j} = \frac{(\Omega_{i-1,j} + \Omega_{i+1,j} + \Omega_{i,j-1} + \Omega_{i,j+1})}{4}$$



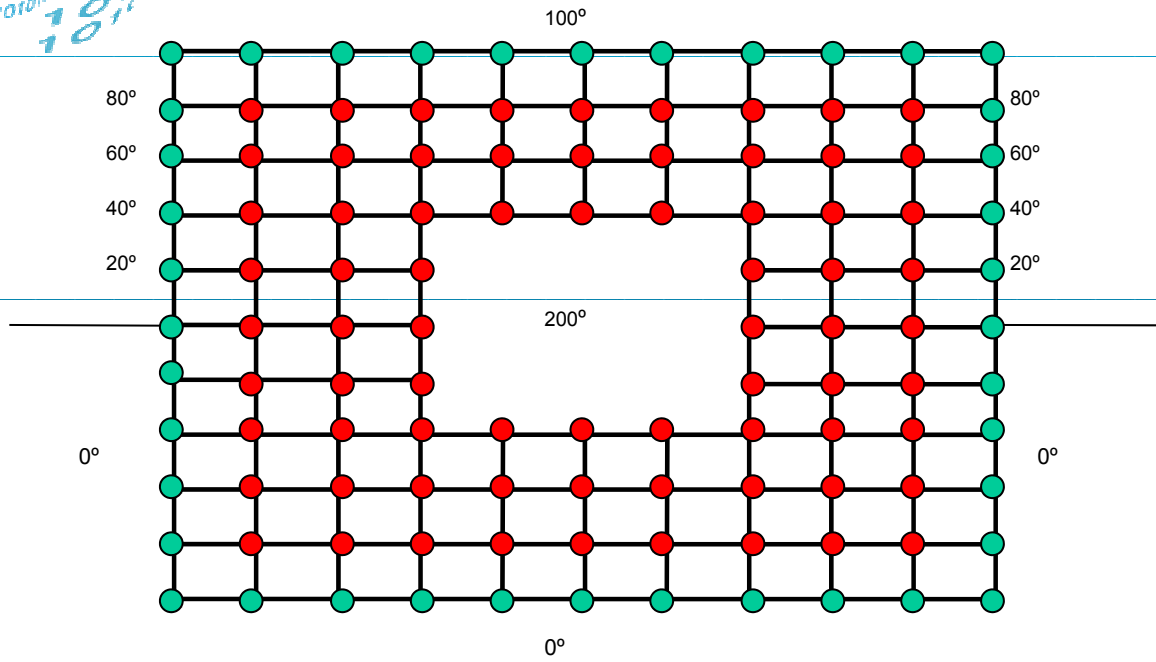
SISTEMAS DE ECUACIONES LINEALES

Por un tubo de secciones interior y exterior cuadradas, circula un líquido a temperatura de 200° C. El tubo está parcialmente sumergido en hielo, de manera que la mitad inferior del exterior del tubo está a una temperatura de 0 ° C. La cara superior del tubo se mantiene a 100° C. Se supone que la temperatura en los laterales varía linealmente entre los 0° C de la parte superior de hielo y los 100° C de la cara superior. La sección interior mide 4 cm. De lado y la exterior 10 cm. La distribución de la temperatura en una sección transversal del tubo se aproxima mediante una malla con nudos cada cm.

- Resolver el sistema por un método directo
- Estudiar la convergencia de los métodos indirectos de Jacobi y Gauss-Seidel.
- Hallar la solución por otro método numérico que acelere la convergencia.
- Mostrar la estructura del sistema resultante.
- Presentar una solución con una herramienta computacional (se recomienda MATLAB)
- Dibujar las isotermas en la sección del tubo estudiada.



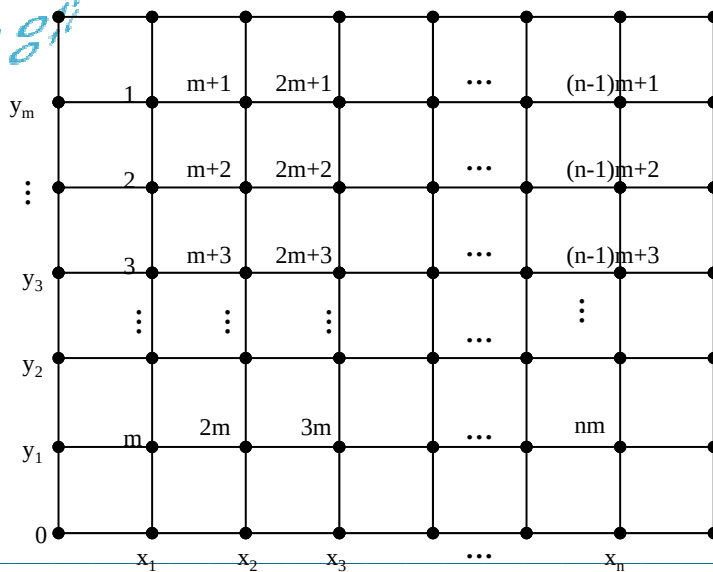
PROBLEMA 'PROPUESTO



- TEMPERATURA CONOCIDA (40 nodos)
- TEMPERATURA DESCONOCIDA (72 nodos)



Algoritmo de Distribución de Temperaturas



**Malla de
arriba-abajo y de
izquierda-derecha**

Resultado de la discretización de la ecuación de

LAPLACE, $\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = 0$ donde $u(x, y)$ es la función a obtener

"u" es la temperatura en un punto (x, y) del plano



Algoritmo de Distribución de Temperaturas

Denotemos por $u_{i,j}$ el valor de la temperatura "u" en el punto $(x_i, y_j) = (ih, jh)$.

$$\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = 0$$

$$i = 0, 1, 2, \dots, n+1$$

$$j = 0, 1, 2, \dots, m+1$$

La derivada parcial segunda de "u" con respecto a "x" dos veces en (x_i, y_j) se aproxima por diferencias finitas como:

$$\frac{\partial^2 u(x_i, y_j)}{\partial x^2} \approx \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h^2}$$

$$\frac{\partial^2 u(x_i, y_j)}{\partial y^2} \approx \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h^2}$$

Substituyendo en la ecuación de Laplace estas ecuaciones para cada nodo del interior de la malla, se obtiene un sistema de ecuaciones lineales con "nm" incógnitas:



Algoritmo de Distribución de Temperaturas

Denotemos por $u_{i,j}$ el valor de la temperatura "u" en el punto $(x_i, y_j) = (ih, jh)$.

$$u_{i-1,j} + u_{i,j} - 1 - 4u_{i,j} + u_{i,j+1} + u_{i+1,j} = 0$$

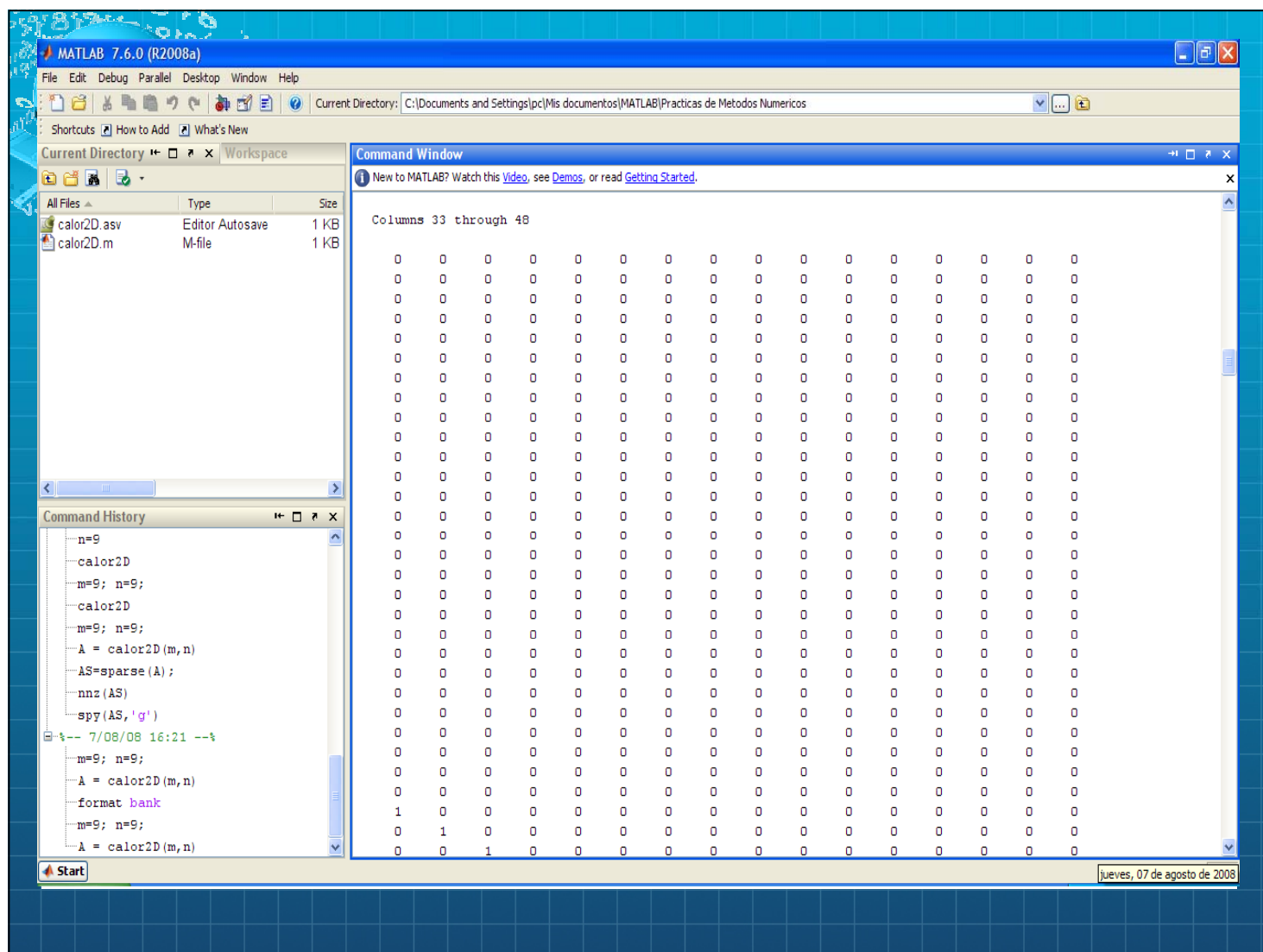
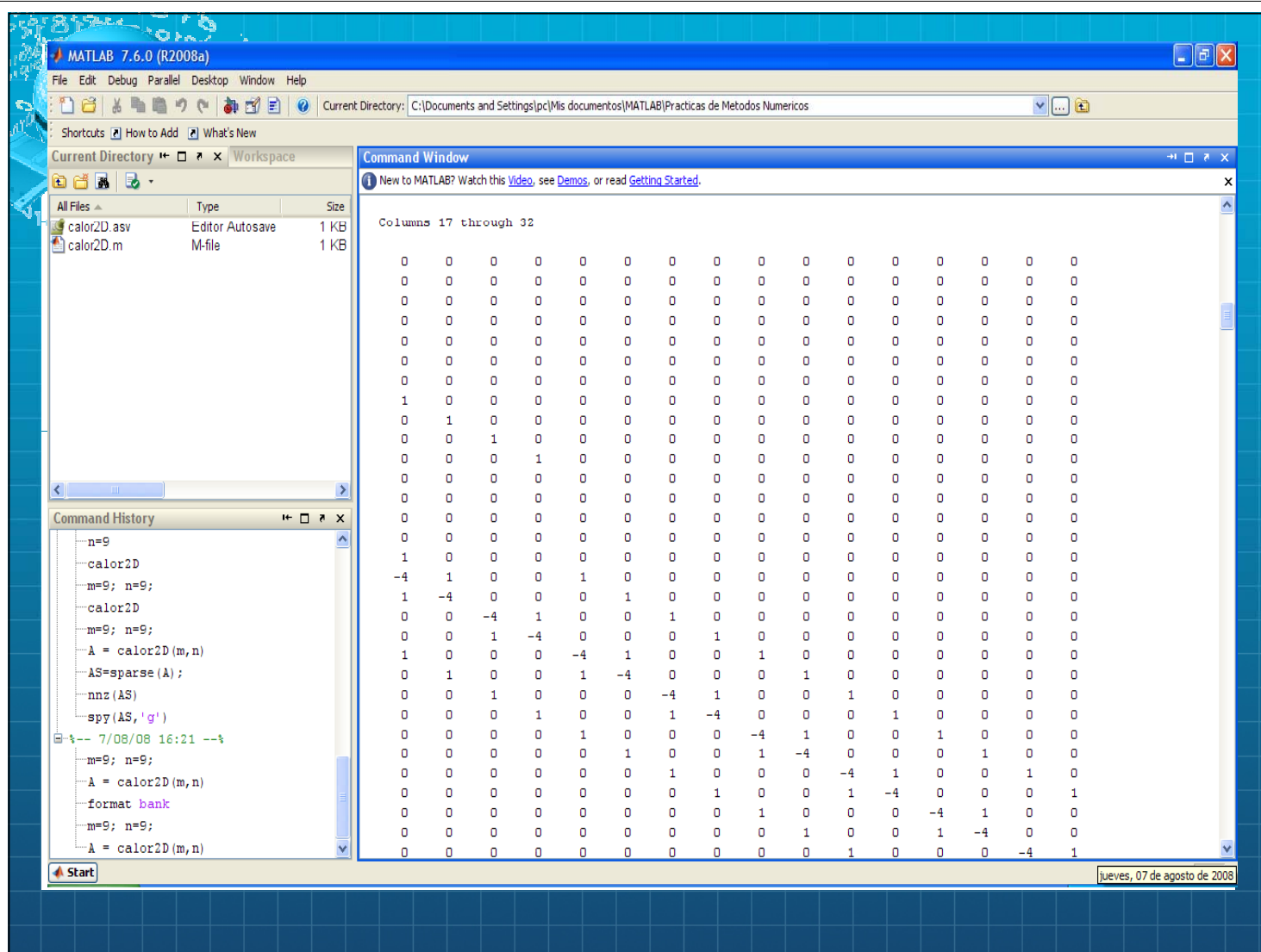
$$i = 0, 1, 2, \dots, n+1$$

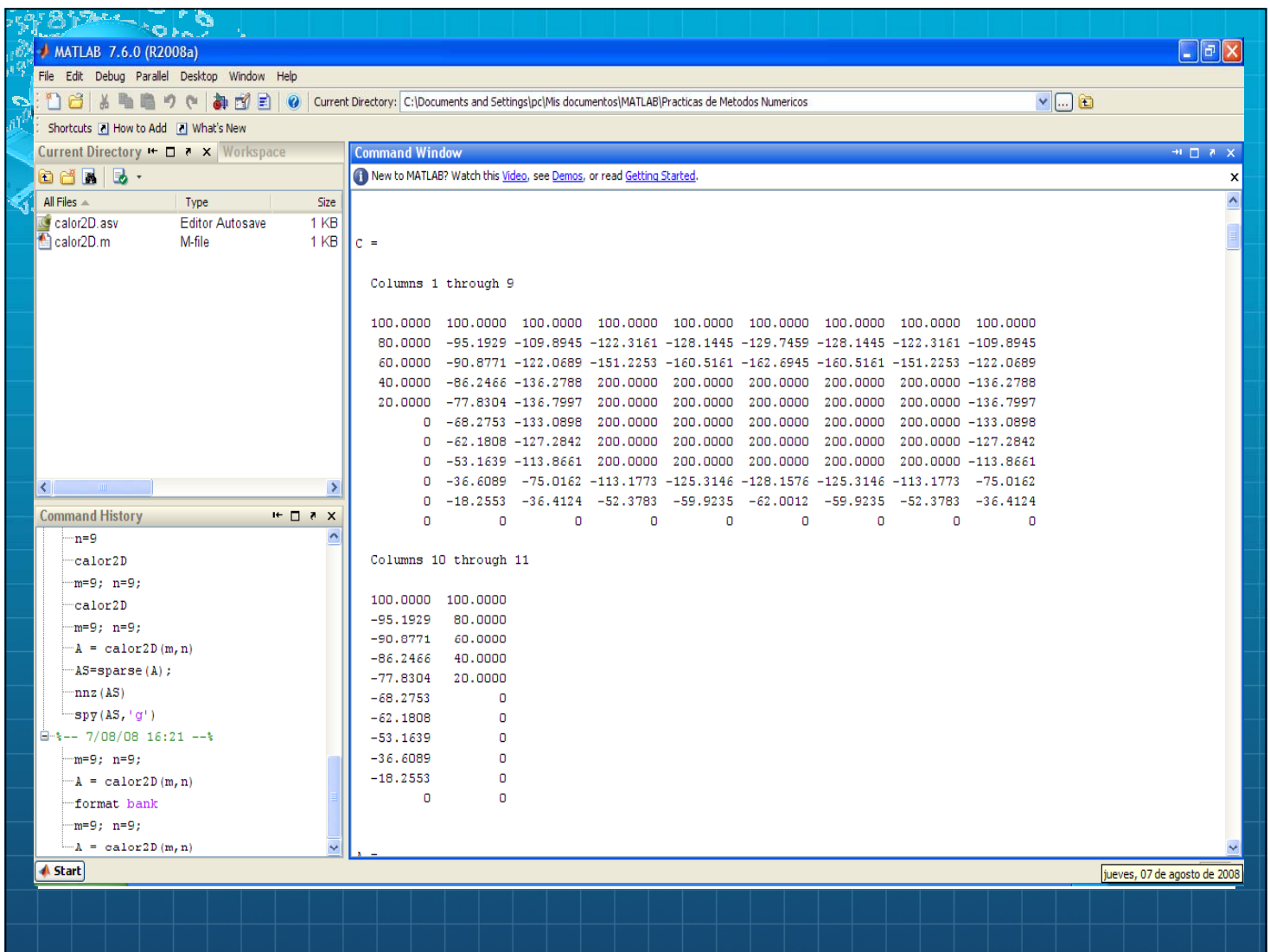
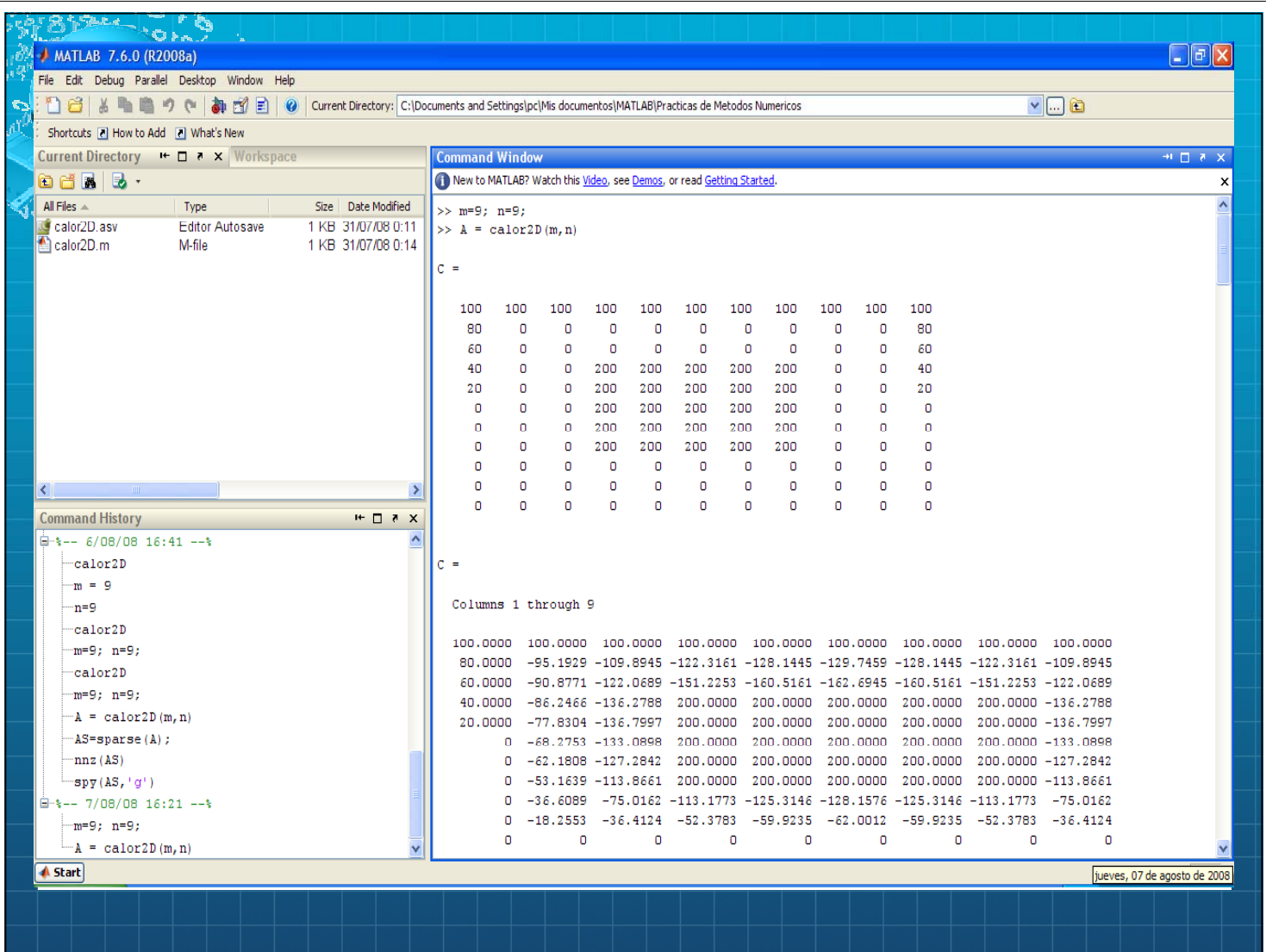
$$j = 0, 1, 2, \dots, m+1$$

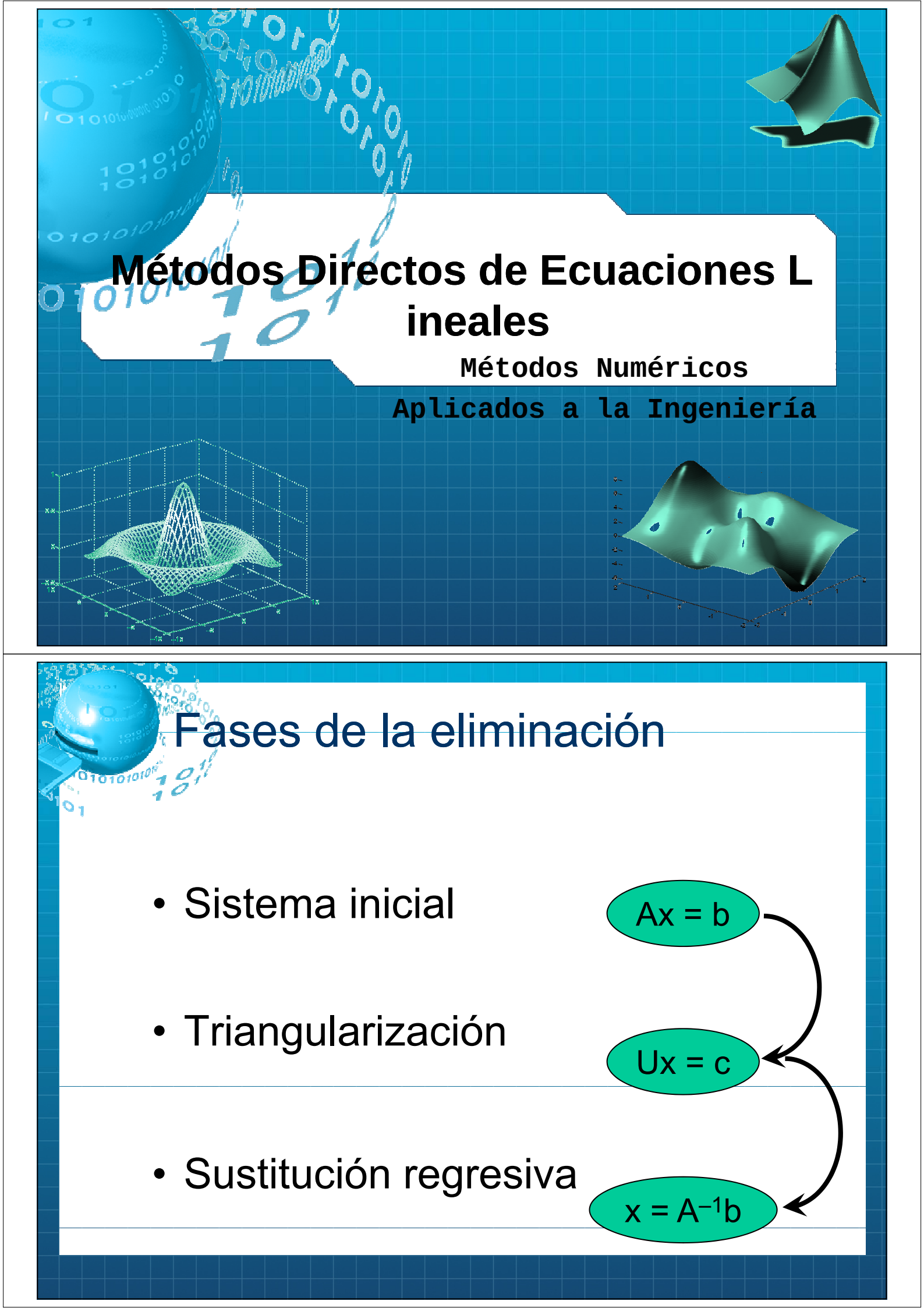
Los términos independientes de este sistema se obtienen de las condiciones de frontera.

La función de MATLAB siguiente construye por diagonales la matriz correspondiente a una malla $m \times n$

```
function A = calor2D(m,n)
p = m*n;
v = ones(1, p-1); % Diagonales 1 y -1
v(m:m:p-m) = 0;
w = ones(1,p-m); % Diagonales m y -m
A = -4*eye(p)+ diag(v,1)+ diag(v,-1) + diag(w,m)+ diag(w,-m);
```

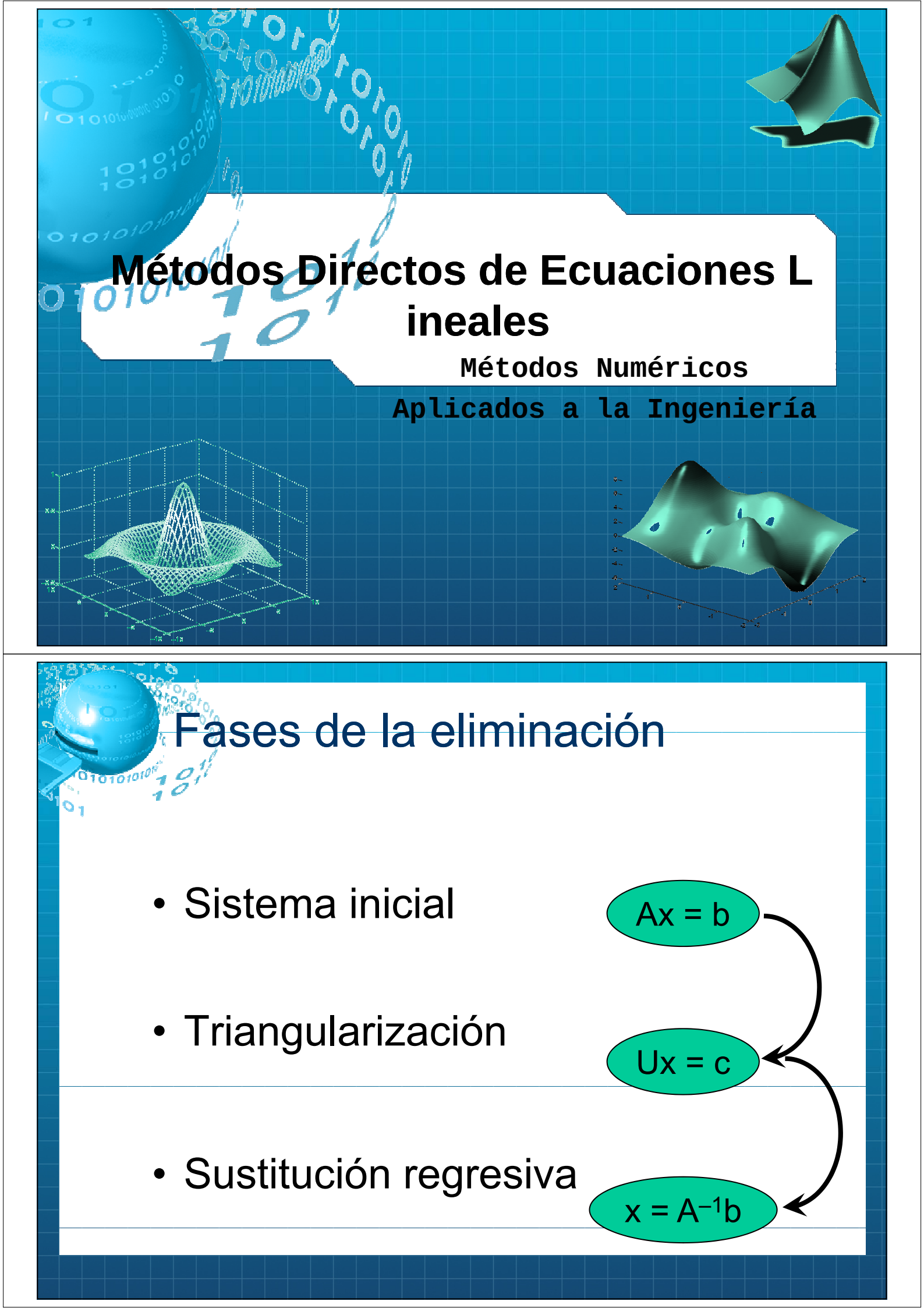
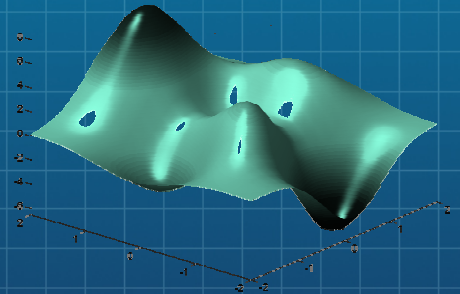
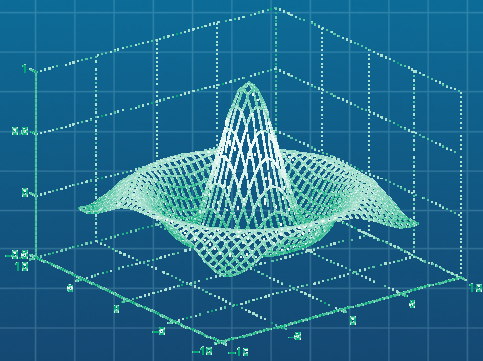







Métodos Directos de Ecuaciones Lineales

Métodos Numéricos
Aplicados a la Ingeniería



Fases de la eliminación

- Sistema inicial
- Triangularización
- Sustitución regresiva

$$Ax = b$$

$$Ux = c$$

$$x = A^{-1}b$$



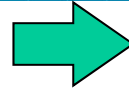
Factorización LU

Sistema original

$$\begin{array}{l} Ax = b \\ LUx = \end{array}$$

Sistemas triangulares

$$\begin{array}{l} Ly = b \\ Ux = y \end{array}$$



» $[L, U] = \text{lu}(a)$

» $[L, U, P] = \text{lu}(a)$

- ☐ Resolución de múltiples sistemas con la misma matriz.
- ☐ Inversa por el método de Jordan-Gauss



CASOS DE APLICACIÓN

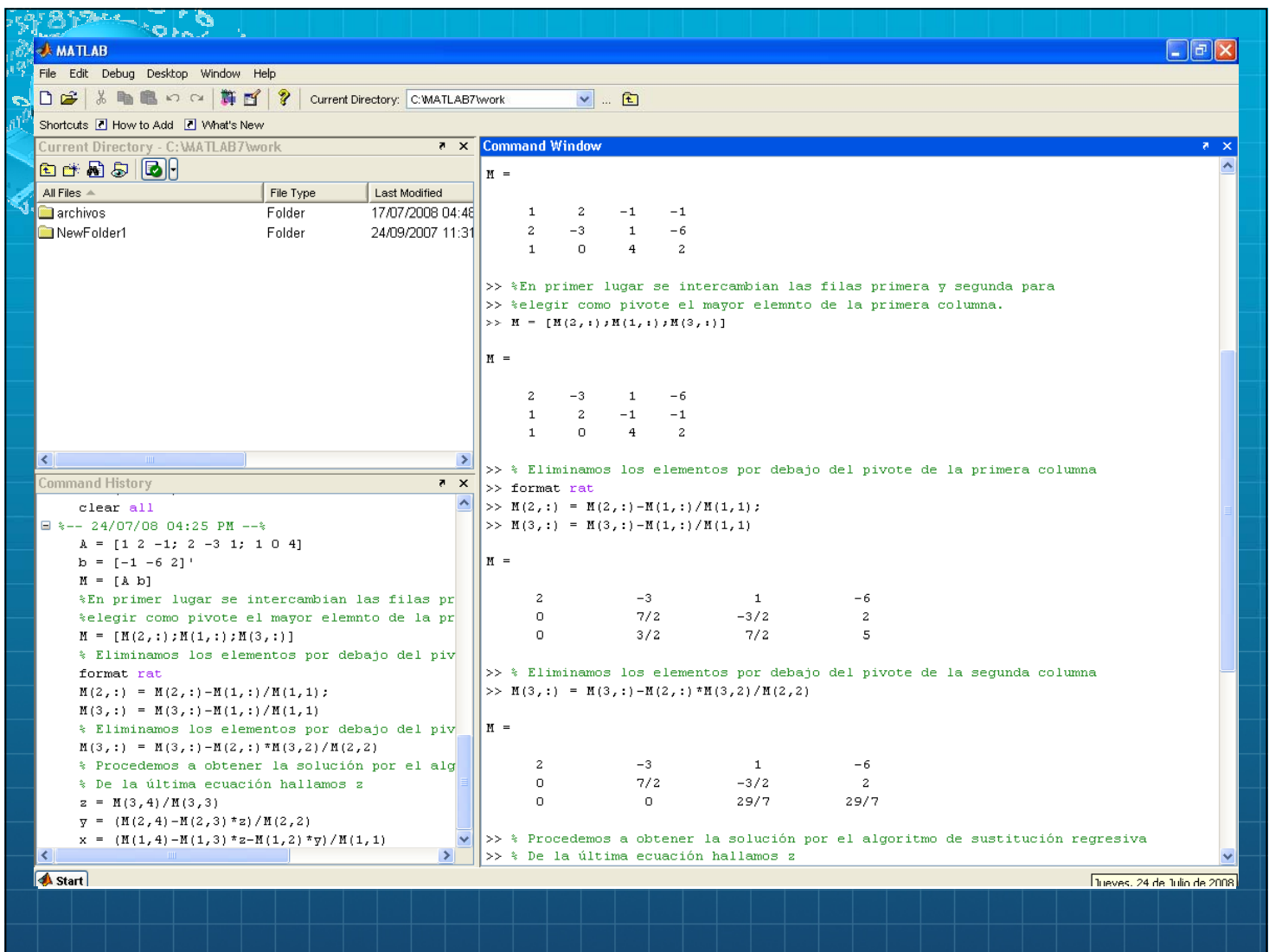
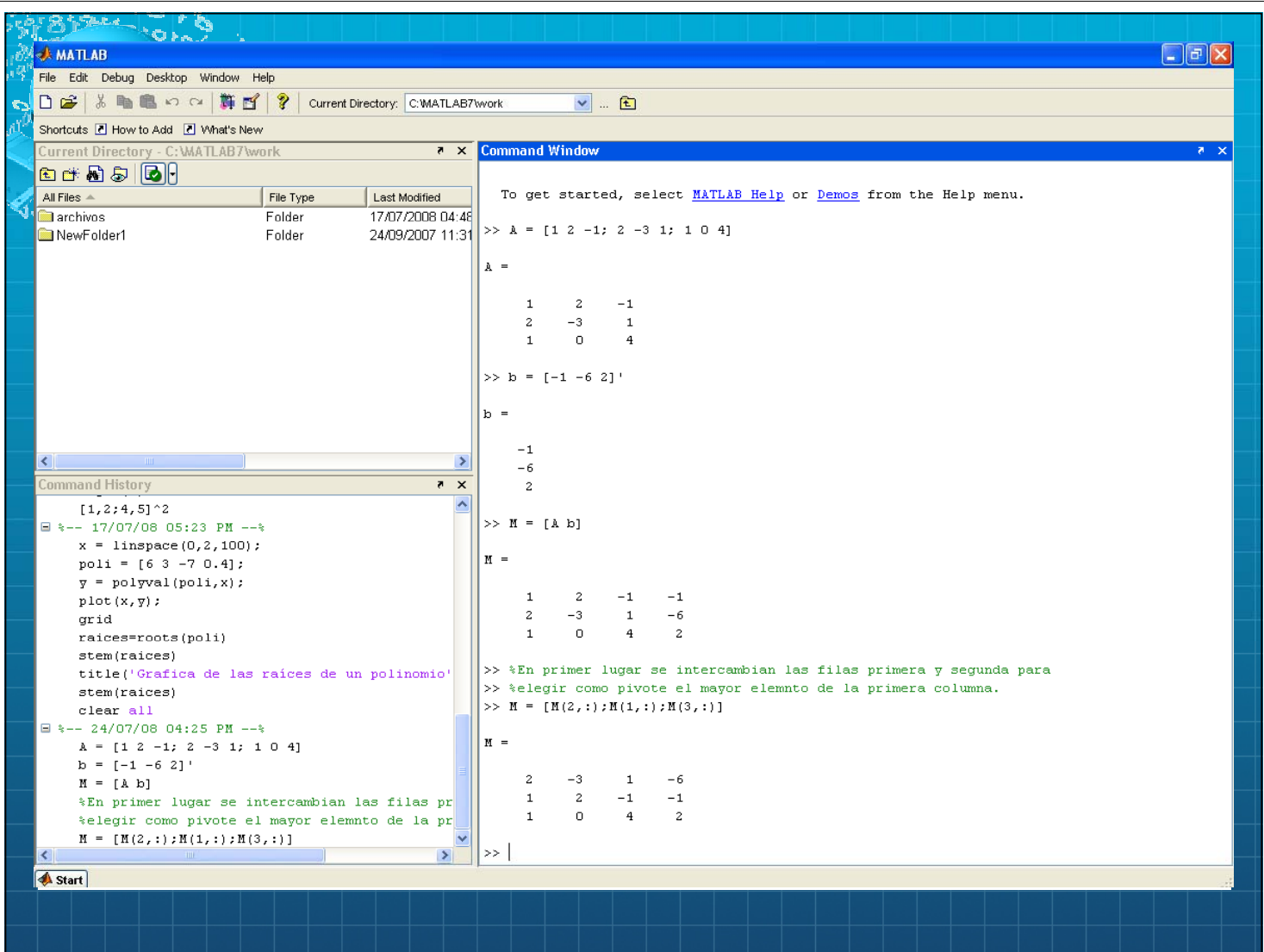
Problema N° 01

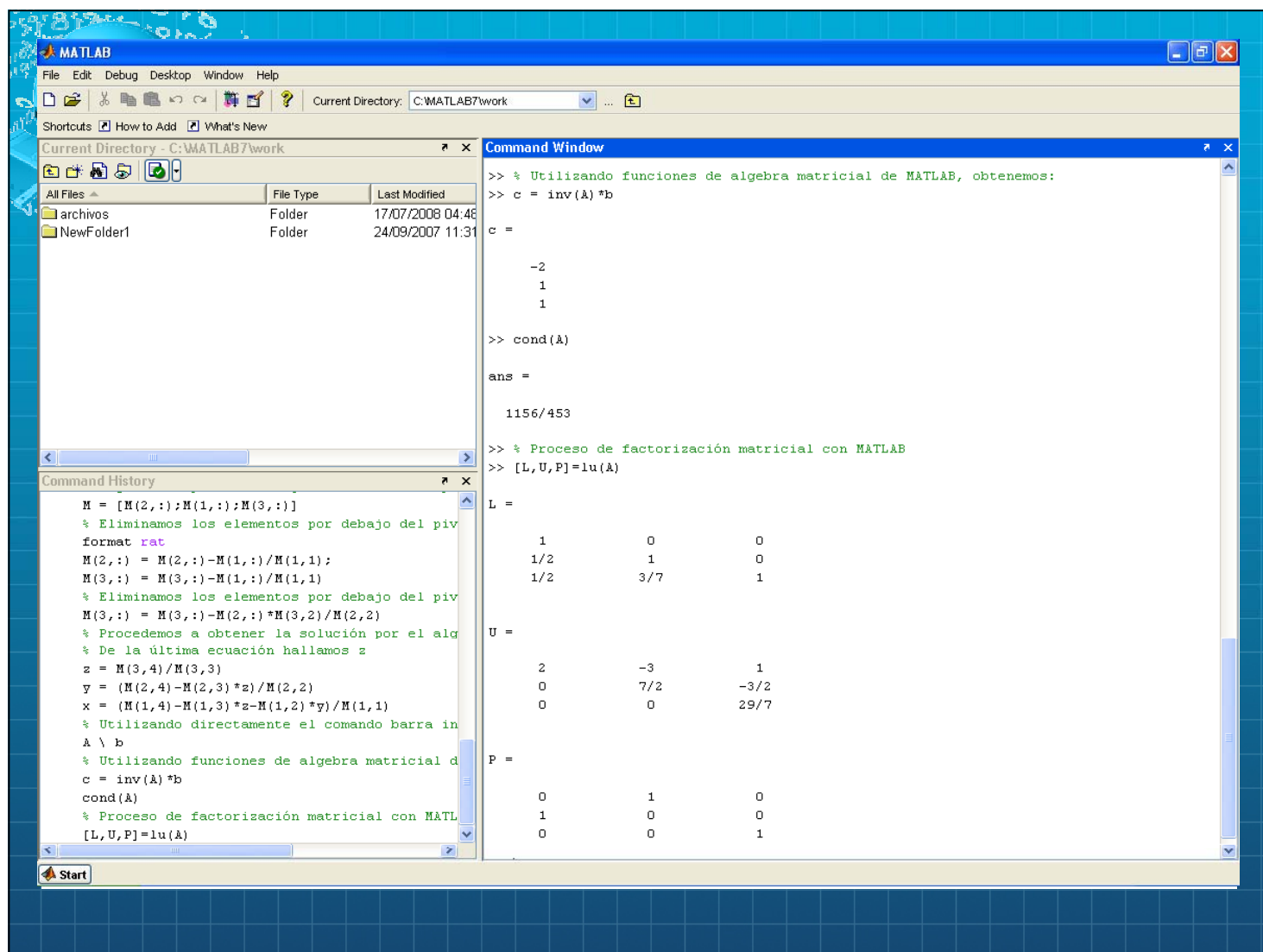
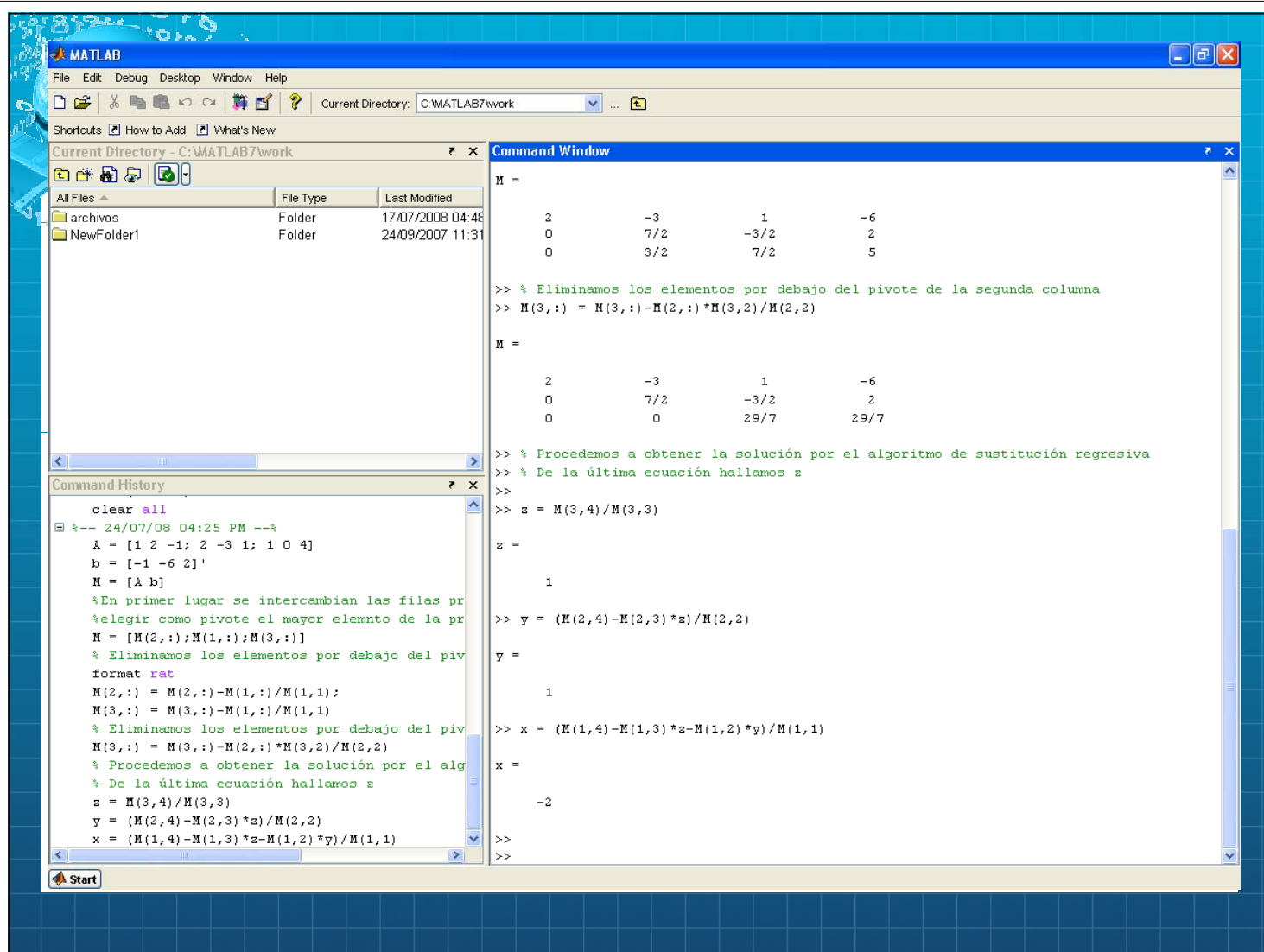
Aplicar el método de eliminación de Gauss con pivotación parcial para resolver el siguiente sistema lineal, operando directamente en MATLAB con notación matricial:

$$\begin{array}{rcl} x + 2y - z & = & -1 \\ 2x - 3y + z & = & -6 \\ x + 4z & = & 2 \end{array}$$

Comprobar la obtención de la solución directamente con el operador `\` de MATLAB

Expresar matricialmente las operaciones elementales realizadas en el proceso, de tal manera que se obtenga la factorización LU de la matriz del sistema

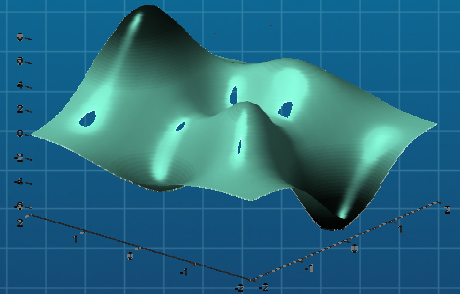
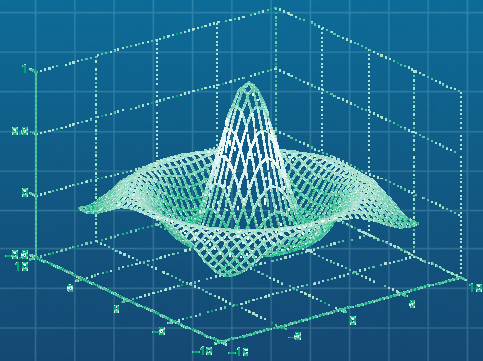






Matrices Dispersas

Métodos Numéricos
Aplicados a la Ingeniería



Matrices Dispersas

- Creación de matrices dispersas
 - `sparse(A)`
 - `full(a)`
 - `speye(n)`
- Operaciones usuales
 - `+` `-` `*` `\` `lu`
- Otras funciones de MATLAB
 - `issparse(A)`
 - `spy(a)`



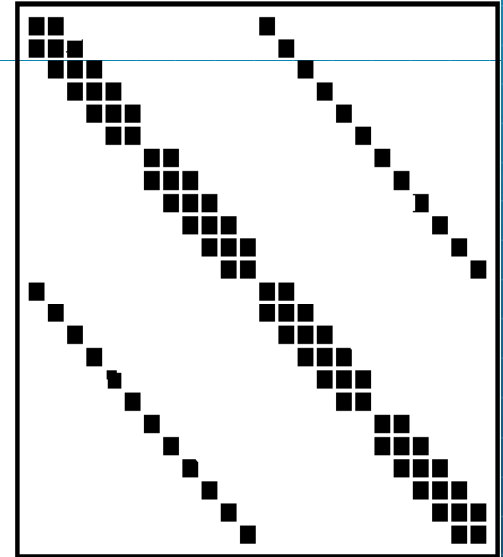
Ejemplo práctico

- Las matrices dispersas aparecen en múltiples aplicaciones.

Como resultado de la discretización de la ecuación de Poisson,

$$-\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) = f \quad \text{en } \Omega \quad \text{y } u = 0 \quad \text{en } \Gamma$$

Aparecen matrices de forma muy similar a la de la figura.



Matrices dispersas en MATLAB

Características:

- Almacenamiento interno mediante formato coordinado
- El acceso es idéntico al de las matrices densas. MATLAB encapsula el formato coordinado.
- El formato coordinado utilizado por MATLAB es dinámico.
- Generalmente, sobre las matrices dispersas se pueden utilizar las mismas funciones que sobre las densas, pero existen excepciones.
- Se pueden utilizar los operadores elementales (+, -, *, /), con operandos densos y dispersos. Si un operador es denso y otro disperso el resultado es denso.



Manejo básico de matrices dispersas en MATLAB (I)

- AS es una matriz dispersa de dimensiones 5x5. si se teclea su nombre en la línea de órdenes aparece un listado de sus elementos.

```
>> AS
```

```
(1,1) 1.3  
(5,1) 6.8  
(2,2) 2.4  
(3,3) 1.4  
(4,4) 7.1  
(1,5) 8.2  
(5,5) 9.5
```

- ➔ se muestra cada elemento, con los correspondientes índices de fila y columna.
- ➔ Los elementos se hallan ordenados por columnas.



Manejo básico de matrices dispersas en MATLAB (II)

- La matriz **AS** del ejemplo anterior se puede generar, a partir de una densa, utilizando la función **sparse**.

```
>> A=[1.3, 0, 0, 0, 8.2;  
      0, 2.4, 0, 0, 0;  
      0, 0, 1.4, 0, 0;  
      0, 0, 0, 7.1, 0;  
      6.8, 0, 0, 0, 9.5];
```

- ➔ Pese a que ambas matrices son iguales, puesto que poseen los mismos elementos, MATLAB únicamente almacena los elementos no nulos de AS.

```
>> AS=sparse(A);
```

```
>> issparse(A)  
ans=  
0
```

```
>> issparse(AS)  
ans= 1
```

- ➔ La función **issparse** permite saber si una función se halla almacenada o no en formato disperso.



Manejo básico de matrices dispersas en MATLAB (III)

- Se puede acceder y manipular a los elementos de la matriz dispersa **AS** como si de una matriz densa se tratase.

```
>> AS(1,1)  
1.3
```

```
>> AS(3,4)  
0
```

```
>> AS(3,4)=5.5;
```

```
>> AS(1,1)=2.4;
```

```
>> AS
```

- ➡ La posición (3,4) corresponde a un elemento nulo de la matriz AS.
- ➡ Se añade un elemento no nulo.
- ➡ Se modifica un elemento no nulo.
- ➡ tecleando AS podemos ver las modificaciones efectuadas.



Manejo básico de matrices dispersas en MATLAB (IV)

```
>> AS
```

```
AS =
```

```
(1,1) 2.4000
```

```
(5,1) 6.8000
```

```
(2,2) 2.4000
```

```
(3,3) 1.4000
```

```
(3,4) 5.5000
```

```
(4,4) 7.1000
```

```
(1,5) 8.2000
```

```
(5,5) 9.5000
```

- ➡ Se ha modificado.

- ➡ Se ha añadido.



Manejo básico de matrices dispersas en MATLAB (V)

- Se visualiza la matriz **AS** mediante la función **spy**.

```
>> spy(AS,'g')
```



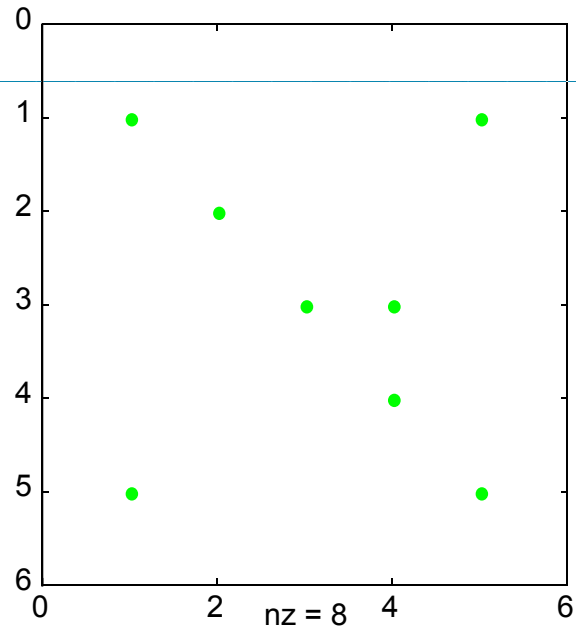
```
>> nnz(AS)
```

```
ans =
```

```
8
```



La función **nnz** permite saber el número de elementos no nulos de **AS**.



Manejo básico de matrices dispersas en MATLAB (VI)

- Si se opera una matriz dispersa con una densa, generalmente, el resultado es denso.

```
>> issparse(AS*A)
```

```
ans =
```

```
0
```

```
>> issparse(AS*ones(5,1))
```

```
ans =
```

```
0
```

- Si se opera una matriz dispersa con otra dispersa, generalmente, el resultado es disperso. Aunque el resultado pueda tener más elementos no nulos que los operandos.

```
>> issparse(AS+AS)
```

```
ans =
```

```
1
```



Manejo básico de matrices dispersas en MATLAB (VII)

- puede ser interesante desmenuzar el formato coordenado, con el fin de manipular la estructura de 3 vectores (i,j,s).

```
>> [i,j,s]=find(AS);  
>> i'
```

➡ La función **find** permite extraer los tres vectores del formato coordenado.

```
ans=  
1 5 2 3 3 4 1 5
```

```
>> j'
```

➡ Los elementos se hallan ordenados por columnas, y en la columna por filas.

```
ans=  
1 1 2 3 4 4 5 5
```

```
>> s'
```

```
ans=  
2.4 6.8 2.4 1.4 5.5 7.1 8.2 9.5
```

```
>> nnz(AS)
```

➡ Lógicamente, el número de elementos no nulos coincide con la longitud de los vectores.

```
ans=  
8
```



Manejo básico de matrices dispersas en MATLAB (VIII)

- La función **sparse** también puede utilizarse para construir una matriz dispersa a partir de los tres vectores del esquema coordenado.

➡ **AS=sparse(i,j,s)**

```
>> BS=sparse(i,j,s);
```

➡ La matriz BS es idéntica a la matriz AS.

- Las funciones **tril** y **triu** permiten separar el triángulo inferior y superior de una matriz dispersa.

```
>> BSL=tril(BS);
```

```
>> BSU=triu(BS);
```



Manejo básico de matrices dispersas en MATLAB (IX)

- La función **full** realiza la operación inversa de la función **sparse**. convierte una matriz dispersa en densa.

```
>> A=full(AS)
```



La matriz A es almacenada en formato denso, pese a que su estructura es dispersa.

A =

2.4	0	0	0	8.2
0	2.4	0	0	0
0	0	1.4	5.5	0
0	0	0	7.1	0
6.8	0	0	0	9.5



Manejo básico de matrices dispersas en MATLAB (y X)

- La función **speye** permite generar una matriz dispersa identidad.
- La función **sparse** permite, también, generar una matriz dispersa vacía

```
» speye(5)
```

ans =

(1,1)	1
(2,2)	1
(3,3)	1
(4,4)	1
(5,5)	1

```
» sparse(5,5)
```

ans =

All zero sparse: 5-by-5

.



Generación aleatoria de matrices dispersas (I)

- La función **sprand** permite generar matrices dispersas uniformemente distribuidas.
- ➔ **S=sprandn(m,n,density)** es una matriz dispersa de dimensión $m \times n$ y con, aproximadamente, $\text{density} \cdot m \cdot n$ elementos no nulos.

```
>> AS=sprandn(50,50,0.1);  
>> spy(AS);  
>> nnz(AS)  
ans=
```

244

```
>> 50*50*0.1
```

```
ans=
```

250

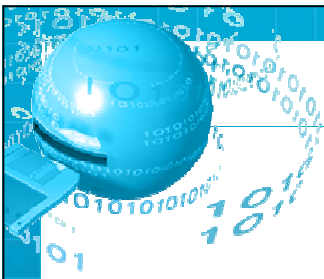
```
>> AS=sprandn(50,50,0.5);  
>> spy(AS);  
>> nnz(AS)  
ans=
```

974

```
>> 50*50*0.5
```

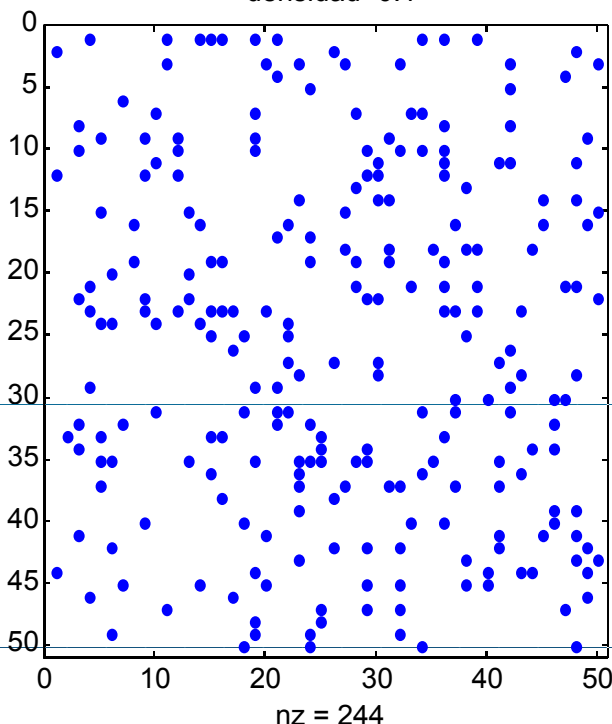
```
ans=
```

1250

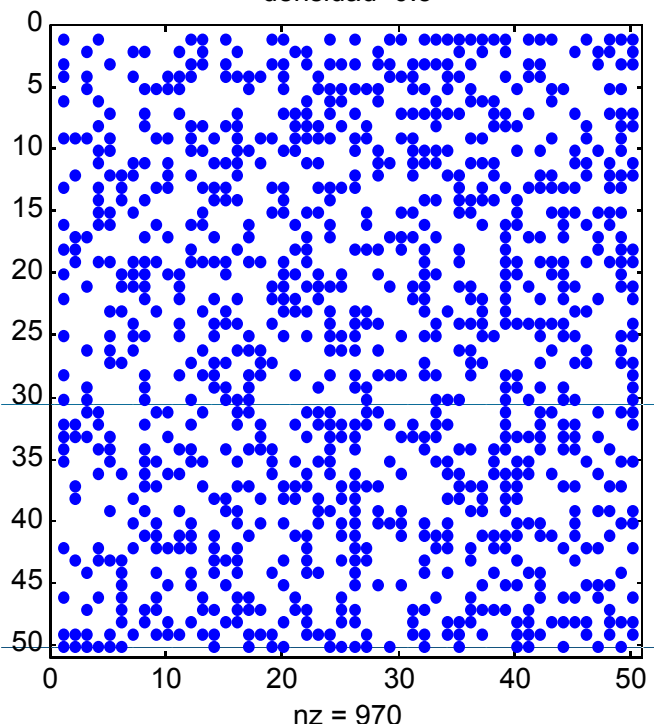


Generación aleatoria de matrices dispersas (II)

densidad=0.1



densidad=0.5





Generación aleatoria de matrices dispersas (III)

➔ **S=sprandn(m,n,density,rc)** es una matriz dispersa que, además, tiene un número de condición próximo a **1/rc**.

>> AS=sprandn(50,50,0.1,0.1);	>> AS=sprandn(50,50,0.1,0.01);
>> spy(AS);	>> spy(AS);
>> nnz(AS)	>> nnz(AS)
ans=	ans=
242	245
>> condest(AS)	>> condest(AS)
ans=	ans=
53.945	1502.6



Generación aleatoria de matrices dispersas (IV)

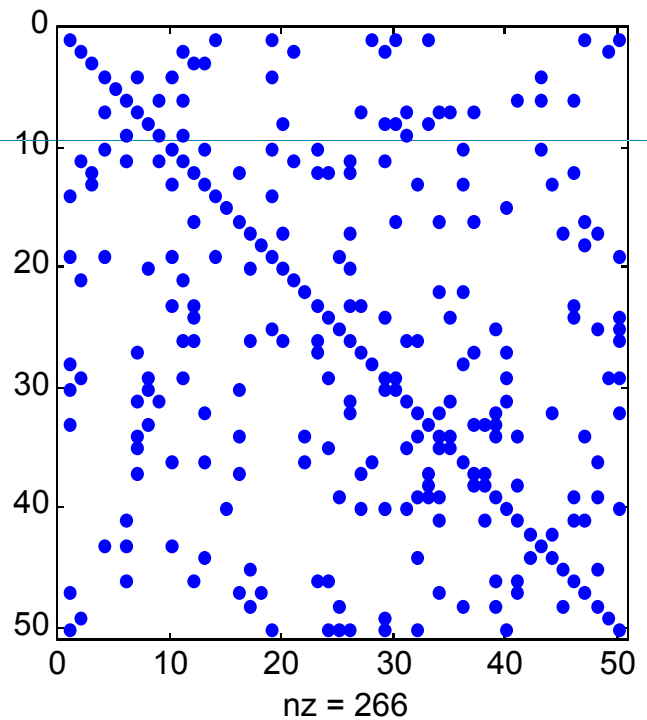
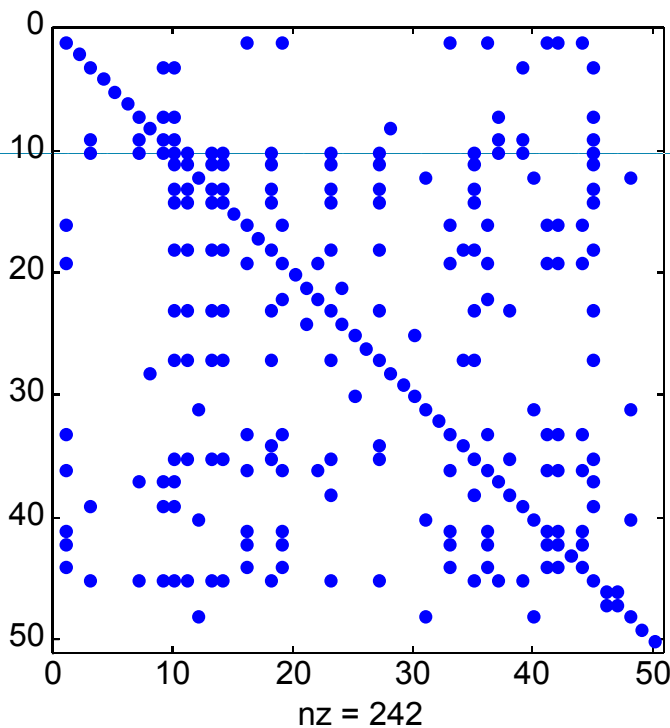
➔ **S=sprandsym(n,density,rc)** es una matriz dispersa simétrica.

➔ **S=sprandsym(n,density,rc,kind)** es una matriz dispersa simétrica definida positiva. **kind=1** o **2**.

>>AS=sprandsym(50,0.1,0.1,1);	>>AS=sprandsym(50,0.1,0.01,1);
>>spy(AS);	>>spy(AS);
>>nnz(AS)	>>nnz(AS)
ans=	ans=
258	266
>>condest(AS)	>>condest(AS)
ans=	ans=
14.7642	665.0635



Generación aleatoria de matrices dispersas (y V)



Resumen de funciones (I)

1.- Matrices Dispersas Elementales.

- speye - Matriz Identidad Dispersa.
- sprandn - Matriz Aleatoria Dispersa.
- sprandsym - Matriz Aleatoria Simétrica Dispersa.
- spdiags - Matrices Dispersas Banda formadas a partir de diagonales. .

2.- Conversion de Matrices densas a dispersas.

- sparse - Crea una matriz dispersa a partir de la información de ciertos índices.
- full - Convierte una matriz en formato disperso a formato denso.
- find - Encuentra los índices de las entradas no nulas.



Resumen de funciones (II)

Funciones que trabajan con los elementos no nulos de la matriz

nnz	- Número de entradas no nulas
nonzeros	- Elementos no nulos
nzmax	- Aumento de almacenamiento reservado para los elementos no nulos.
spones	- Reemplaza las entradas no nulas por unos.
issparse	- Devuelve 1 si es dispersa, 0 en otro caso.
spfun	- Aplica la función a los elementos no nulos.



Resumen de funciones (III)

Visualización de matrices dispersas

spy	- Visualiza la estructura de la matriz dispersa
gplot	- Dibuja un grafo, "graph theory".

Reordering algorithms.

colmmd	- Mínimo grado por columnas.
symmmd	- Mínimo grado Simétrico.
symrcm	- Ordenación Cuthill-McKee inversa.
colperm	- Ordenación por columnas basada en los elementos no nulos.
randperm	- Vector permutación.



De todo esto se deduce la necesidad de estudiar la implementación eficiente del producto matriz por vector utilizando formatos de almacenamiento dispersos


$$\begin{aligned} b_1 &= a_{1j}x_i \\ b_2 &= a_{2j}x_i \\ b_3 &= a_{3j}x_i \\ b_4 &= a_{4j}x_i \\ b_5 &= a_{5j}x_i \end{aligned}$$

J



Operaciones Matriciales Básicas.

Producto matriz vector disperso

Filas disperso ($Ax=b$)

0. $b=(0,0,0,\dots,0)^t$
1. *para* $i=1:nfil$
2. *para* $j=I(i):I(i+1)-1$
3. $b_i=b_i+S(j)*x_{J(j)}$
4. **Fin**
5. **.fin**

Filas denso ($Ax=b$)

0. $b=(0,0,0,\dots,0)^t$
1. *para* $i=1:nfil$
2. *para* $j=1:ncol$
3. $b_i=b_i+a_{ij}*x_j$
4. **fin**
5. **.fin**

$$\begin{bmatrix} 0 & 0 & 0 & a & 0 & b \\ 0 & c & 0 & 0 & d & 0 \\ 0 & 0 & 0 & e & 0 & 0 \\ 0 & f & 0 & 0 & 0 & 0 \\ 0 & 0 & g & 0 & h & 0 \\ i & 0 & 0 & 0 & 0 & j \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \end{bmatrix}$$

nnz
1 2 3 4 5 6 7 8 9 10

S: [a b c d e f g h i j]

J: [4 6 2 5 4 2 3 5 1 6]

I: [1 3 5 6 7 9 11]

1 2 3 4 5 6 7
nfil+1



Operaciones Matriciales Básicas.

Producto matriz vector disperso

$$\begin{bmatrix} 0 & 0 & 0 & a & 0 & b \\ 0 & c & 0 & 0 & d & 0 \\ 0 & 0 & 0 & e & 0 & 0 \\ 0 & f & 0 & 0 & 0 & 0 \\ 0 & 0 & g & 0 & h & 0 \\ i & 0 & 0 & 0 & 0 & j \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \end{bmatrix}$$

nnz
1 2 3 4 5 6 7 8 9 10

S: [a b c d e f g h i j]

J: [4 6 2 5 4 2 3 5 1 6]

I: [1 3 5 6 7 9 11]

1 2 3 4 5 6 7
nfil+1

1. for $i=1:nfil$	2. for $j=I(i):I(i+1)-1$	3. $b_i=b_i+S(j)*x_{J(j)}$	
$i=1$	$j=1:2 \Rightarrow$ $I(1):I(2)-1$	$b_1 = a x_4 + b x_6$ $S(1) x_{J(1)} \quad S(2) x_{J(2)}$ $j=1 \quad j=2$	
$i=2$	$j=3:4 \Rightarrow$ $I(2):I(3)-1$	$b_2 = c x_2 + d x_5$ $S(3) x_{J(3)} \quad S(4) x_{J(4)}$ $j=3 \quad j=4$	
...			
$i=6$	$j=9:10 \Rightarrow$ $I(6):I(7)-1$	$b_6 = i x_1 + j x_6$ $S(9) x_{J(9)} \quad S(10) x_{J(10)}$ $j=9 \quad j=10$	



Ejercicio

Producto matriz vector disperso orientado a columnas ($Ax=b$)

`function [b] =mv_csc(I,J,S,x)`

$$\begin{bmatrix} 0 & 0 & 0 & a & 0 & b \\ 0 & c & 0 & 0 & d & 0 \\ 0 & 0 & 0 & e & 0 & 0 \\ 0 & f & 0 & 0 & 0 & 0 \\ 0 & 0 & g & 0 & h & 0 \\ i & 0 & 0 & 0 & 0 & j \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \end{bmatrix}$$

$$j=1: \quad b_6 = b_6 + i \, x_1$$

$$j=2: \quad b_2 = b_2 + c \, x_2 \\ b_4 = b_4 + f \, x_2$$



Ejercicio

`function [y]= mv_csc (I, J, S, x)`

`cols=length(J)-1;`

`fiels=max(I);`

`y=zeros(fiels,1);`

`for j = 1:cols`

`for k=J(j): J(j+1)-1`

`i=I(k);`

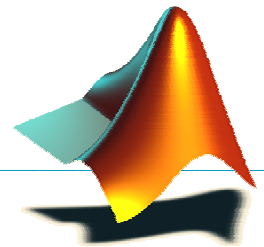
`y(i) = y(i) + x(j)*S(k);`

`end`

`end`



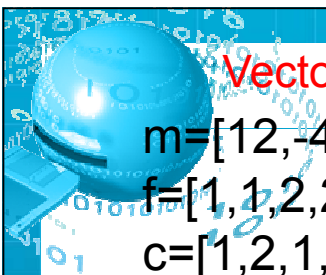
Vectores y matrices



Ejemplo 1: Tomemos la siguiente matriz

$$\begin{pmatrix} 12 & -4 & 0 & 0 & 0 \\ 7 & 3 & 0 & 0 & -8 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -13 & 11 & 0 \\ 0 & 0 & 2 & 7 & -4 \end{pmatrix}$$

Y trabajemos con ella como una matriz dispersa, para ello definimos el vector de elementos no nulos, el vector definido por las filas y el vector definido por las columnas



Vectores y matrices

```
m=[12,-4,7,3,-8,-13,11,2,7,-4];
```

```
f=[1,1,2,2,2,4,4,5,5,5];
```

```
c=[1,2,1,2,5,3,4,3,4,5];
```

```
a=sparse(f,c,m,5,5)
```

```
full(a)
```

```
m1=[12,7,-4,3,-13,2,11,7,-8,-4];
```

```
f1=[1,2,1,2,4,5,4,5,2,5];
```

```
c1=[1,1,2,2,3,3,4,4,5,5];
```

```
b=sparse(f1,c1,m1,5,5);
```

```
full(b)
```

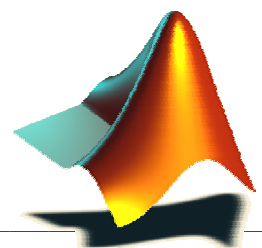
```
a=sparse(f,c,m)
```

```
s=a+b
```

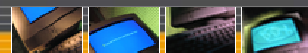
```
p=a*b
```

```
[f2,c2,m2]=find(p)
```

```
e=full(sparse(f2,c2,m2))
```



BIBLIOGRAFÍA RECOMENDADA



ACADEMIA DE ANÁLISIS NUMÉRICO

Elementos de Métodos Numéricos para Ingeniería
Editorial MCGRAW HILL. Universidad de Guadalajara. México. 2002

AKAI, TERRENCE J.

Métodos Numéricos Aplicados a la Ingeniería (Applied Numerical Methods for Engineers)
John Wiley & Sons, INC. - Editorial Limusa, Grupo Noriega Editores. México. 2000

BATHE, K.J. y WILSON, E.L.

Numerical Methods in Finite Element Analysis
Edit. Prentice Hall Inc., Englewood Cliffs, N.J.

CHAINSKAIA, LUDMILA - DOIG, ELIZABETH

Elementos de Análisis Numérico - Algoritmos y Aplicaciones
Fondo Editorial Pontificia Universidad Católica del Perú. Lima. 1999

CHAPRA, STEVEN C. - CANALE RAYMOND P.

Métodos Numéricos Para Ingenieros
Editorial MCGRAW HILL. México. 1998

GERALD C.F. - WHEATLEY P.O.

Análisis Numérico con Aplicaciones
Editorial PRENTICE HALL - Pearson Educación. 6ta. Edición. México. 2000

MEYER CARL D.

Matrix Analysis and Applied Linear Algebra
Published by SIAM. EE.UU. 2000

NAKAMURA, SHOICHIRO

Análisis Numérico y Visualización Gráfica con MATLAB
Editorial PEARSON EDUCATION - México. 2001

NIEVES, ANTONIO - DOMÍNGUEZ, FEDERICO

Métodos Numéricos Aplicados a la Ingeniería
Editorial CECSA. México. 1995

PIKE - GUERRA

Optimización en Ingeniería
Editorial ALFAOMEGA- 1ra. Edición. México. 1989

PRESS W.H. - TEUKOLSKY S.A. - VETTERLING W.T. - FLANNERY B.P.

Numerical Recipes in C: The Art of Scientific Computing
Published by the Press Syndicate of the University of Cambridge. Second Edition. EE.UU. 1992

RAFFO LECCA, Eduardo

Algoritmos: Análisis y Diseño
Editorial Raffo-Lecca. Lima. Perú. 1999

RAFFO LECCA, Eduardo

Métodos Numéricos Para Ciencias e Ingeniería con Pascal
Editorial Raffo-Lecca. Lima. Perú. 1997

SCALETTI FARINA, HUGO

Métodos Numéricos Avanzados
Universidad Nacional de Ingeniería. FIC-CISMID. Lima. Perú. 2001

SEGEWICK ROBERT

Algorithms
ADDISON-WESLEY Publishing Company - Brown University. EE.UU. 1983

TIRUPATHI R. CHANDRUPATLA, ASHOK D. BELEGUNDU

Introducción al Elemento Finito en Ingeniería
Editorial Prentice Hall - Addison Wesley Longman. México. 1999

ZIENKIEWICZ, O.C. - TAYLOR R.L.

El Método de los Elementos Finitos - Volumen 1: Formulación Básica y Problemas Lineales
Editorial MCGRAW HILL - CIMNE Barcelona. 4ta. Edición. España. 1993

ZIENKIEWICZ, O.C. - TAYLOR R.L.

El Método de los Elementos Finitos - Volumen 2: Mecánica de Sólidos y Fluidos. Dinámica y No Linealidad

